RMM

RMM System

Complete Handbook

Remote Monitoring & Management Platform

Version 1.0 · NinjaOne-style, built in-house

All staff: technicians · administrators · managers · developers

Table of Contents

RMM System — Complete Handbook

Remote Monitoring and Management Platform Version 1.0 — NinjaOne-style, built in-house

Prepared for: All staff — new joiners, IT support, technicians, administrators, managers, and developers **System URL:** http://localhost:8501 **API URL:** http://localhost:5000 **Document version:** 2.0 (comprehensive edition)

Foreword

This handbook is the single, authoritative reference for everyone who uses, operates, or maintains the RMM system. It begins where the system begins — with installation — and walks through every feature, every screen, and every action you will ever need to perform.

This document is written in plain language. Technical jargon is explained when first introduced. Whether you are setting up the server for the first time, handling your first support ticket, or performing a monthly billing run, the relevant chapter will tell you exactly what to do and why.

Who should read what:

Role	Start Here	Then Read
System installer / IT admin setting up for first time	Part I (Installation)	Parts II, VII
New employee — first week on the job	Part II (Getting Started)	Part III, Chapter 15
IT Support staff (helpdesk, tickets)	Part II, Part III, Chapter 15	Part IV (Management)
Technician (devices, patches, scripts)	Parts II–VI	Part VIII (Technical)
Manager (reports, billing)	Parts II, VII	Part III overview
Administrator (users, system health)	All parts	Part VIII
Developer / maintainer	Parts I, VIII	Everything

Callout conventions used throughout:

> **NOTE**
>
> Background information or extra context.

> **TIP**

A smarter or faster way to do something.

WARNING

Something that can cause problems or is hard to reverse.

IMPORTANT

Information you must not skip.

Table of Contents

PART I — INSTALLATION AND SETUP

Chapter 1: System Requirements

Who uses this chapter

The person responsible for installing and hosting the RMM system — typically a senior technician, IT administrator, or the developer who built the system.

What you need before you begin

This chapter describes the hardware and software requirements for the machine that will **host** the RMM system (the server). This is not the managed client machine — those requirements are in Chapter 6.

Minimum Hardware (RMM Server)

Component	Minimum	Recommended
Processor	2 cores, 2.0 GHz	4 cores, 3.0 GHz+
RAM	4 GB	8 GB
Disk Space	20 GB free	50 GB+ (for logs, reports, database growth)
Network	100 Mbps LAN	1 Gbps LAN + internet access
OS	Windows 10 (64-bit)	Windows 10/11 or Windows Server 2019/2022

Required Software

The following software must be installed on the RMM server before the application will run:

Software	Version	Purpose
Python	3.11 or later	Runs the API, dashboard, and agent
PostgreSQL	14 or later	Primary database
Redis (Memurai for Windows)	6.x or later	Background task queue
pip	Included with Python	Python package installer
Git (optional)	Any recent	For cloning the repository

Network Ports Used

Port	Service	Direction
8501	Streamlit Dashboard	Inbound from users' browsers
5000	Flask API	Inbound from dashboard and agents
5432	PostgreSQL	Localhost only (internal)
6379	Redis/Memurai	Localhost only (internal)

> **NOTE**
>
> If agents will connect from remote networks, port 5000 must be accessible through any firewall between the RMM server and those networks.

> **WARNING**
>
> This system is designed for internal/LAN deployment. Do not expose port 5000 or 8501 to the public internet without first adding a reverse proxy (e.g., nginx) with HTTPS.

Chapter 2: Installing Prerequisites

Step 1: Install Python 3.11+

1. Open a browser and go to **python.org/downloads**.
2. Download the latest Python 3.11.x or 3.12.x Windows installer (64-bit).
3. Run the installer.
4. On the first screen, check the box **"Add Python to PATH"**. This is essential.
5. Click **Customize installation**.
6. Ensure **pip** is checked.
7. On the Advanced Options screen, check **"Install for all users"**.
8. Click **Install**.
9. When complete, click **Disable path length limit** if prompted.
10. Open PowerShell and verify the installation:

```
python --version
pip --version
```

Both commands should display version numbers. If they display errors, Python was not added to PATH — reinstall and ensure step 4 is completed.

Step 2: Install PostgreSQL

1. Go to **postgresql.org/download/windows** and download the installer for PostgreSQL 14, 15, or 16.
2. Run the installer.

3. Accept all defaults for the installation directory.
4. When prompted for a **superuser password**, set a strong password for the `postgres` account. **Write this down** — you will need it in the next step.
5. Leave the default port as **5432**.
6. Leave the default locale.
7. Complete the installation. Uncheck "Launch Stack Builder" at the end.

Create the RMM database and user:

8. Open the **SQL Shell (psql)** application that was installed with PostgreSQL.
9. Press Enter to accept all defaults at the prompts until you reach the password prompt.
10. Enter the superuser password you set in step 4.
11. At the `postgres=#` prompt, run these commands exactly as shown (type each line and press Enter):

```
CREATE USER rmm_app WITH PASSWORD 'your_secure_password_here';
CREATE DATABASE rmmdb OWNER rmm_app;
GRANT ALL PRIVILEGES ON DATABASE rmmdb TO rmm_app;
\q
```

Replace `your_secure_password_here` with a strong password of your choice. **Write this down** — you will need it in Chapter 4.

> **NOTE**
>
> The PostgreSQL Windows service starts automatically after installation. You can verify it is running in **Windows Services** (search "Services" in Start menu) — look for "postgresql-x64-14" or similar.

Step 3: Install Redis via Memurai

Memurai is a Redis-compatible server designed for Windows. It is the recommended Redis implementation for this system on Windows.

1. Go to **memurai.com/get-memurai** and download the free Developer edition installer.
2. Run the installer. Accept all defaults.
3. Memurai installs as a Windows service and starts automatically.
4. Verify Redis is running — open PowerShell and run:

```
Test-NetConnection -ComputerName localhost -Port 6379
```

You should see `TcpTestSucceeded : True`.

> **TIP**
>
> If you already have an existing Redis installation or use another Redis-compatible server, you can use that instead. The only requirement is a Redis server accessible at localhost:6379 (or update the REDIS_URL environment variable accordingly).

Chapter 3: Setting Up the RMM Application

Step 1: Obtain the Application Files

If you have the project as a ZIP file:

1. Copy the ZIP to a suitable directory, e.g., `C:\RMM\`.
2. Right-click the ZIP → **Extract All** → extract to `C:\RMM\RemoteManagementSystem\`.

If you are cloning from a Git repository:

```
git clone <repository-url> C:\RMM\RemoteManagementSystem
```

The resulting directory structure should look like:

```
RemoteManagementSystem\
■■■ api\
■■■ agent\
■■■ dashboard\
■■■ scripts_library\
■■■ build_pdf.py
■■■ CLAUDE.md
```

Step 2: Install API Dependencies

1. Open PowerShell and navigate to the `api` folder:

```
Set-Location C:\RMM\RemoteManagementSystem\api
```

2. Create a Python virtual environment:

```
python -m venv venv
```

3. Activate the virtual environment:

```
.\venv\Scripts\Activate.ps1
```

Your prompt should now show `(venv)` at the beginning.

4. Install all required packages:

```
pip install -r requirements.txt
```

This will install Flask, SQLAlchemy, Celery, JWT, bcrypt, and all other API dependencies. It may take 2–3 minutes.

> **WARNING**
>
> If PowerShell shows "execution of scripts is disabled", run this command first: `Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser`

Step 3: Install Dashboard Dependencies

1. Open a second PowerShell window and navigate to the `dashboard` folder:

```
Set-Location C:\RMM\RemoteManagementSystem\dashboard
```

2. Create and activate a virtual environment:

```
python -m venv venv
.\venv\Scripts\Activate.ps1
```

3. Install dependencies:

```
pip install -r requirements.txt
```

This installs Streamlit, Plotly, Pandas, and requests.

Step 4: Install Agent Dependencies

On the RMM server (you can also do this on each managed machine later):

1. Navigate to the `agent` folder:

```
Set-Location C:\RMM\RemoteManagementSystem\agent
```

2. Create and activate a virtual environment:

```
python -m venv venv
.\venv\Scripts\Activate.ps1
```

3. Install dependencies:

```
pip install -r requirements.txt
```

This installs psutil, requests, pywin32, wmi, and schedule.

Chapter 4: First-Time Configuration

Creating the API Environment File

The API requires a configuration file called `.env` inside the `api\` directory. This file holds sensitive settings such as the database password and secret keys.

1. In the `api\` folder, create a new file named `.env` (note: no extension, just `.env`).
2. Open it with a text editor (Notepad, VS Code, etc.) and paste the following template:

```
SECRET_KEY=replace-with-a-long-random-string-at-least-32-chars
DATABASE_URL=postgresql://rmm_app:your_secure_password_here@localhost:5432/rmmdb
JWT_SECRET_KEY=replace-with-another-long-random-string-at-least-32-chars
ORG_REGISTRATION_TOKEN=replace-with-a-unique-org-token-for-agent-registration
REDIS_URL=redis://localhost:6379/0
CELERY_BROKER_URL=redis://localhost:6379/0
CELERY_RESULT_BACKEND=redis://localhost:6379/1
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=
SMTP_PASSWORD=
SMTP_FROM=RMM System <noreply@yourcompany.com>
```

3. Make the following substitutions:

Placeholder	Replace with
`replace-with-a-long-random-string-at-least-32-chars`	A random string of 32+ characters (e.g., `d7f3a1c9e8b2f4d6a7c3e9f1b5d8a2c4`)
`your_secure_password_here`	The PostgreSQL password you set in Chapter 2, Step 2
`replace-with-another-long-random-string-at-least-32-chars`	A different random string of 32+ characters
`replace-with-a-unique-org-token-for-agent-registration`	A unique token agents use to register (e.g., `rmm-prod-2026-companyname-abc123`)

> **IMPORTANT**
>
> The `ORG_REGISTRATION_TOKEN` is what prevents unauthorized agents from registering with your RMM server. Use a long, unique, hard-to-guess value. You will need this same token when configuring the agent's `config.ini` file in Chapter 6.

> **WARNING**
>
> Never commit the `.env` file to version control (Git). It contains secrets. The `.gitignore` file already excludes it.

Configuring Email Notifications (Optional)

If you want the system to send email alerts when critical thresholds are crossed, fill in the SMTP settings:

- For Gmail: use `smtp.gmail.com`, port `587`, and your Gmail address. For the password, use a Google **App Password** (not your regular Gmail password) — generate one at **myaccount.google.com/apppasswords**.
- For Microsoft 365: use `smtp.office365.com`, port `587`.
- For a local SMTP relay: use your relay server's hostname and port.

If left blank, the system operates normally but sends no email notifications.

Initializing the Database

With the virtual environment activated in `api\`:

1. Run the database seeder to create all tables and the first admin user:

```
Set-Location C:\RMM\RemoteManagementSystem\api
.\venv\Scripts\Activate.ps1
python seed.py
```

2. The seed script will:
   - Create all database tables (devices, users, tickets, alerts, etc.)
   - Create a default admin user
   - Create a sample default customer
   - Seed the 7 built-in PowerShell scripts into the script library
3. The output will display the **admin email and password** created. Note these immediately — this is your first login credential.

> **TIP**
>
> If the seed script fails with a database connection error, verify that PostgreSQL is running and that the `DATABASE_URL` in your `.env` file contains the correct password and database name.

Chapter 5: Starting All Services

The RMM system consists of five separate processes that must all be running for full functionality. Each runs in its own terminal window.

Service Startup Order

Always start services in this order. Starting them out of order can cause startup errors.

```
1. PostgreSQL (Windows Service — starts automatically)
2. Redis/Memurai (Windows Service — starts automatically)
3. Flask API
4. Celery Worker
5. Celery Beat
6. Streamlit Dashboard
7. Agent (on each managed machine)
```

Starting the Flask API (Terminal 1)

```
Set-Location C:\RMM\RemoteManagementSystem\api
.\venv\Scripts\Activate.ps1
python app.py
```

Expected output:

```
* Running on http://0.0.0.0:5000
* Debug mode: on
```

Leave this terminal open. Do not close it — closing it stops the API.

Starting the Celery Worker (Terminal 2)

```
Set-Location C:\RMM\RemoteManagementSystem\api
.\venv\Scripts\Activate.ps1
celery -A tasks.celery_app worker --pool=solo -l info
```

Expected output: Lines showing `[celery@HOSTNAME] ready.`

> **NOTE**
>
> The `--pool=solo` flag is required on Windows. Celery's default pool type (`prefork`) is not compatible with Windows.

Starting Celery Beat (Terminal 3)

```
Set-Location C:\RMM\RemoteManagementSystem\api
.\venv\Scripts\Activate.ps1
celery -A tasks.celery_app beat -l info
```

Expected output: Lines showing `beat: Starting...` and scheduled tasks being logged.

Celery Beat is the scheduler. It triggers automatic tasks such as evaluating alert rules every 60 seconds and running scheduled automation profiles.

Starting the Dashboard (Terminal 4)

```
Set-Location C:\RMM\RemoteManagementSystem\dashboard
.\venv\Scripts\Activate.ps1
streamlit run app.py
```

Expected output:

```
You can now view your Streamlit app in your browser.
Local URL: http://localhost:8501
Network URL: http://192.168.x.x:8501
```

Open your browser and go to **http://localhost:8501**. You should see the RMM login page.

Verifying All Services Are Running

Open a new PowerShell window and run:

```
netstat -ano | findstr ":5000 :8501 :5432 :6379"
```

You should see lines for each port, confirming all services are listening.

Quick Health Check via Browser

1. Open: **http://localhost:5000/api/health**
2. You should see a JSON response like: `{"status": "ok", "database": "connected", "redis": "connected"}`
3. If any service shows "disconnected", check that respective service is running.

Chapter 6: Deploying the Agent on Managed Machines

The agent is a lightweight Python program that runs on every machine you want to monitor. It sends metrics to the API and receives commands from it.

What You Need on Each Managed Machine

- Windows 10 or Windows 11 (64-bit)
- Python 3.11+ with pip
- Network access to the RMM server on port 5000
- Administrator privileges to install and run the agent

Step 1: Install Python on the Managed Machine

Follow the same steps as Chapter 2, Step 1. Verify with:

```
python --version
```

Step 2: Copy the Agent Files

Copy the entire `agent\` folder from the RMM server to the managed machine. You can use a USB drive, a network share, or any other transfer method.

Suggested destination on the managed machine: `C:\RMM\agent\`

Step 3: Install Agent Dependencies

On the managed machine, open PowerShell as **Administrator**:

```
Set-Location C:\RMM\agent
python -m venv venv
.\venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

Step 4: Configure the Agent

Open `C:\RMM\agent\config.ini` in a text editor. You will see:

```
[api]
url = http://localhost:5000
org_token =

[agent]
device_id =
agent_token =
heartbeat_interval = 60
software_interval = 21600
version = 1.0.0

[logging]
level = INFO
file = rmm_agent.log
```

Make these changes:

1. `url` — change `http://localhost:5000` to the actual IP address or hostname of your RMM server.
   Example: `http://192.168.1.100:5000`
2. `org_token` — enter the `ORG_REGISTRATION_TOKEN` value. Find it in either:

• The dashboard: **Admin** → **System Info** tab → **Agent Enrollment Token** card (click Reveal)
• Or directly in the `.env` file on the server: `ORG_REGISTRATION_TOKEN=...`

3. Leave `device_id` and `agent_token` blank — the agent fills these in automatically on first registration.

Example of a completed config:

```
[api]
url = http://192.168.1.100:5000
org_token = rmm-prod-2026-companyname-abc123

[agent]
device_id =
agent_token =
heartbeat_interval = 60
software_interval = 21600
version = 1.0.0

[logging]
level = INFO
file = rmm_agent.log
```

Step 5: Run the Agent

On the managed machine, open PowerShell as **Administrator** and run:

```
Set-Location C:\RMM\agent
.\venv\Scripts\Activate.ps1
python rmm_agent.py
```

The first time it runs, the agent will register itself with the RMM server and display:

```
Device registered successfully. Device ID: <uuid>
Starting heartbeat loop...
```

After this, the agent sends a heartbeat every 60 seconds. You can now go to the RMM dashboard and see this machine appear in the Devices page.

Step 6: Set Up Auto-Start (Optional but Recommended)

To ensure the agent restarts automatically when the machine reboots:

1. Create a batch file `start_agent.bat` in `C:\RMM\agent\`:

```
@echo off
cd C:\RMM\agent
call venv\Scripts\activate.bat
python rmm_agent.py
```

2. Add this batch file to Windows Task Scheduler or place a shortcut in the Startup folder (`Win+R` → type `shell:startup`).

> **IMPORTANT**
>
> The agent should run as Administrator for full capability. Task Scheduler can be configured to run the task with highest privileges.

Verifying Agent Registration

1. Go to the RMM dashboard at http://localhost:8501.
2. Log in.
3. Click **Devices** in the sidebar.
4. The newly registered machine should appear with a green online dot.
5. Within 60 seconds, its CPU, RAM, and disk metrics will begin populating.

Deploying the Agent on Other WiFi/LAN Machines

Use this procedure to monitor any Windows, Linux, or macOS machine on the same network — without physically accessing the RMM server machine.

> **NOTE**
>
> The API already binds to `0.0.0.0:5000`, so any machine on the same LAN can reach it. The only change needed is pointing the agent at the server's LAN IP instead of localhost.

Step 1: Get the server's LAN IP

Go to **Admin** → **System Info** tab → **Server IP Addresses** card. Each LAN IP is displayed in a copyable code block. Alternatively, check your router's DHCP client list for the hostname of the RMM server machine.

Step 2: Copy the agent folder to the target machine

Copy the entire `agent\` folder to the target machine using a USB drive, network share, email, or git clone. Suggested destination: `C:\RMM\agent\`

Step 3: Get the org token

Go to **Admin** → **System Info** tab → **Agent Enrollment Token** card → click **Reveal**. Copy the token.

Step 4: Run the setup helper

On the target machine, open PowerShell and run:

```
Set-Location C:\RMM\agent
python -m venv venv
.\venv\Scripts\Activate.ps1
pip install -r requirements.txt
python setup_agent.py 192.168.x.x <org_token>
```

Replace `192.168.x.x` with the server's actual LAN IP and `<org_token>` with the token from Step 3. The script updates `config.ini` automatically and clears any previous device ID so the agent registers fresh.

Step 5: Start the agent

```
python rmm_agent.py
```

Within 60 seconds the device appears in the **Devices** tab with a green online dot.

> **TIP**
>
> Follow Step 6 in this chapter (Task Scheduler auto-start) to ensure the agent restarts after reboots on WiFi machines too.

Chapter 7: First-Time Setup Walkthrough

After all services are running and at least one agent has checked in, complete these setup steps to make the system fully operational.

Step 1: Log In as Administrator

1. Go to **http://localhost:8501**.
2. Log in with the admin credentials output by `seed.py` in Chapter 4.
3. You should land on the RMM home page showing the stat cards.

Step 2: Create Your First Customer

Every device must belong to a customer. Even if you are managing your own organization's devices, you still need a customer record.

1. Click **Customers** in the sidebar.
2. Click **+ New Customer**.
3. Enter your organization or first client's name.
4. Fill in contact email and phone.
5. Select the appropriate tier (standard, premium, or enterprise).
6. Click **Save**.

Step 3: Assign Devices to the Customer

1. Click **Devices** in the sidebar.
2. Find the newly registered device (it will appear with no customer assigned initially).
3. Click the device to expand it.
4. Assign it to the customer you just created.

Step 4: Create Your First Alert Rules

Without alert rules, no alerts will ever fire — the system needs rules to know when something is wrong.

1. Click **Alerts** in the sidebar.
2. Click the **Alert Rules** tab.
3. Create at minimum these four rules (full instructions in Chapter 13):

• CPU critical: metric=cpu, operator=gt, threshold=90, severity=critical, cooldown=15
• Disk critical: metric=disk, operator=gt, threshold=90, severity=critical, cooldown=60
• RAM critical: metric=ram, operator=gt, threshold=90, severity=critical, cooldown=15
• Device offline: metric=offline, severity=warning, cooldown=60

Step 5: Create Additional User Accounts

1. The `Admin` → `Users` tab shows all current users.
2. Use the create user form to add accounts for each team member.
3. Assign the appropriate role: **admin** for administrators, **technician** for IT staff, **viewer** for managers or clients.
4. Share credentials with each user and instruct them to change their password after first login.

Step 6: Create Your First Automation Profile

A basic automation profile ensures devices are regularly maintained without manual intervention.

1. Click **Automation** in the sidebar.
2. Click **Create / Edit Profile**.
3. Name it "Weekly Standard Maintenance".
4. Set schedule: weekly, Sunday, 02:00.
5. Enable OS patches (critical + security only), Restore Point, and Delete Temp Files.
6. Save the profile.

The system is now fully operational. The following parts of this handbook cover day-to-day usage.

PART II — GETTING STARTED

Chapter 8: What is the RMM System?

What it is

RMM stands for Remote Monitoring and Management. An RMM system is a platform used by IT teams to monitor, manage, and support computers and servers — often for multiple clients — from a single central dashboard.

Think of it like a control tower for an airport. Every aircraft (your managed devices) sends constant status updates. The tower (the RMM) watches all of them, raises alarms when something is wrong, and allows controllers (your IT staff) to take action remotely.

This system is modeled after industry tools like NinjaOne and ConnectWise Automate. It is composed of four main components:

1. The Dashboard (Streamlit frontend) A web-based interface running at http://localhost:8501. This is what you see when you open the system in your browser. It has 16 pages covering everything from device monitoring to billing.

2. The API (Flask backend) Running at http://localhost:5000, this is the engine that powers the dashboard. It stores and retrieves all data, handles authentication, and processes commands.

3. The Agent (Python) A small program installed on each managed Windows machine. The agent runs continuously, sending a heartbeat to the API every 60 seconds containing:

- CPU usage percentage
- RAM usage percentage
- Disk usage per drive
- System uptime in seconds
- Installed software inventory (every 6 hours)

The agent also receives commands from the API — such as "run this script" or "reboot" — and executes them.

4. The Database (PostgreSQL) All data — devices, tickets, alerts, customers, scripts, patches, billing — is stored in a PostgreSQL database called `rmmdb`.

How it fits your workflow

When a client's machine has a problem — say the hard drive is nearly full — the following chain occurs automatically:

1. The agent reports disk usage of 92% in its heartbeat.
2. The API evaluates this against your alert rules. If a rule says "disk > 90% → critical", an alert is created.

3. The alert appears on the Alerts page and on the Dashboard.
4. An email notification is sent if SMTP is configured.
5. A ticket is automatically created if the alert rule has "auto-create ticket" enabled.
6. A technician investigates, runs a cleanup script, and resolves the ticket.

Steps 1 through 5 happen with zero human involvement. That is the power of an RMM.

Chapter 9: Logging In and Navigation

Who uses this chapter

Everyone. This is the entry point to every page in the system.

Step-by-step: Logging In

1. Open your web browser (Chrome, Firefox, or Edge recommended).
2. Go to: **http://localhost:8501**
3. You will see the RMM login page — dark green background with a centered login card.
4. In the **Email address** field, type your full email address.
5. In the **Password** field, type your password. Characters appear as dots.
6. Click the green **Sign In** → button.
7. Correct credentials take you to the RMM Dashboard.
8. "Invalid credentials" means wrong email or password. Check Caps Lock.

> **NOTE**
>
> Your session token is stored in browser memory only — it is not in the URL. If you share a page URL, the recipient must log in with their own credentials.

> **WARNING**
>
> There is no "forgot password" link. If locked out, contact your system administrator to reset your password.

First Login — Forced Password Change

If an administrator created your account with the **"Require password change on first login"** option enabled, the system will intercept your first login and display a **Set New Password** screen instead of the normal dashboard. You cannot navigate to any other page until you complete the password change.

1. Enter a new password (minimum 8 characters).
2. Confirm the new password.
3. Click **Set Password** →.
4. You are immediately taken to the dashboard. The temporary password no longer works.

Understanding the Interface Layout

After logging in, the screen has two sections:

The Sidebar (left panel) Your navigation menu, always visible. Pages are grouped:

- **MONITORING:** Overview (Dashboard), Devices, Alerts
- **MANAGEMENT:** Tickets, Customers, Automation
- **PATCHING:** OS Patches, Software Patches
- **TOOLS:** Scripts, Disk Management, Maintenance, Network Discovery
- **BUSINESS:** Reports, Billing, Admin

At the top of the sidebar: your name, email, and colored role badge. At the bottom: the **Sign Out** button.

The Main Content Area (right panel) Each page's content appears here. Every page has a title, optional filters, action buttons, and the main data view.

Step-by-step: Navigating Between Pages

1. Look at the sidebar on the left.
2. Find the section heading for the page you want.
3. Click the page name — for example, **Devices** under MONITORING.
4. The right panel loads that page. Your session is maintained as you navigate.

Practical Example: First Login as a New Employee

Sarah is a new junior technician. It is her first day.

1. Her manager gave her credentials: `sarah@company.com` / `Welcome123!`
2. She opens Chrome and goes to http://localhost:8501.
3. She enters her email and password.
4. She clicks **Sign In** →.
5. She sees the RMM Dashboard with "Sarah" displayed at the top of the sidebar alongside a yellow TECHNICIAN badge.
6. She clicks **Devices** to see all managed machines.

Chapter 10: Your Role and What You Can Access

What it is

The system uses Role-Based Access Control (RBAC). Different users have different access depending on their role. There are three roles: **admin**, **technician**, and **viewer**.

The Four Roles Explained

Viewer Read-only access. Viewers can browse the dashboard, look at device metrics, read alerts, and view tickets. They cannot create tickets, run scripts, approve patches, or access the Admin page. Use this role for managers or clients who need read-only visibility.

Technician Full day-to-day operational access. Technicians can:

- Create and update tickets and comments
- Acknowledge and resolve alerts, create alert rules
- View and manage devices
- Run scripts on devices
- Approve and manage patches
- Perform maintenance actions (reboot, shutdown, etc.)
- Manage customers
- Create and edit automation profiles
- Generate reports

Technicians cannot access the Admin page, manage other users, or create invoices.

Administrator Complete access. Includes everything technicians do, plus:

- The Admin page (System Info, Audit Log, Users)
- User management (create, edit, deactivate users)
- Billing and invoice creation
- System-wide configuration

Super Administrator Full system access including emergency recovery. The superadmin account exists permanently — it is re-created automatically every time the API starts if it does not exist. There is exactly one superadmin account per system. It cannot be modified or deleted through the web interface. To change its password, a server administrator must use the CLI tool (`reset_superadmin.py`). See Chapter 26 for details.

Role Badges in the Sidebar

- **SUPERADMIN** — purple badge
- **ADMIN** — red badge
- **TECHNICIAN** — yellow/amber badge
- **VIEWER** — green badge

What Happens If You Exceed Your Role

If a viewer or technician tries to access the Admin page, they see: "Admin access required. This page is restricted to admin users." The page stops loading.

For other restrictions (such as a viewer trying to create a ticket), the API enforces the rule — the action will be rejected with a permission error.

Practical Example: Understanding What You Can Do

Mark is a technician. Monday morning he sees a critical alert.

- View alert details — **YES**
- Click Acknowledge — **YES**
- Click Resolve — **YES**
- Go to Admin page → Audit Log — **NO** (admin only)
- Create a billing invoice — **NO** (admin only)

- Generate a device report — **YES**

PART III — MONITORING

Chapter 11: The Dashboard — Your Command Center

What it is

The Dashboard is the first page you land on after login. It is your at-a-glance health check for the entire system. There are two dashboard views:

1. **The Home Page** (`app.py`) — shows five key stat cards plus a navigation hint. This is what you see immediately after login.
2. **The Overview Page** (`01_Dashboard.py`) — a richer view with charts, a device health map, recent alerts, and an activity feed. Access it by clicking **Overview** in the sidebar.

Who uses it

Everyone. The dashboard is the starting point for all roles.

The Five Stat Cards

Both views display five stat cards across the top:

Card	What it shows
Total Devices	Total registered devices in the system
Online	Devices currently online (heartbeat received recently)
Warning	Devices with at least one metric in the 75–90% range
Critical	Devices with a metric over 90% or with an active critical alert
Open Tickets	Tickets that are not yet resolved or closed

> **TIP**
>
> The colored accent under each card indicates health. Green = normal, amber = caution, red = action required.

The Overview Page — Four Additional Sections

Click **Overview** in the sidebar to see the full dashboard. Below the stat cards:

Device Status Donut Chart (left) A circle chart dividing your fleet into: Healthy, Warning, Critical, and Offline. Hover over a slice to see the exact count.

Device Health Map (right) A grid of mini device cards. Each shows hostname, online/offline status (green dot = online, grey = offline), and current CPU, RAM, and disk percentages. Scan this quickly to spot problems.

Recent Alerts (bottom left) The 10 most recent open alerts. Each row shows a severity color bar, the alert message, and timestamp.

Activity Feed (bottom right) A log of recent system actions — logins, ticket creations, device registrations, alert acknowledgements.

Step-by-step: Morning Health Check

This is the recommended routine to start your shift:

1. Log in at http://localhost:8501.
2. Look at the five stat cards. Note any red numbers.
3. Click **Overview** in the sidebar.
4. Scan the donut chart. If the Critical slice is non-zero, investigate.
5. Scan the Device Health Map for high CPU, RAM, or disk numbers.
6. Read Recent Alerts. Are there critical alerts you have not seen before?
7. Glance at the Activity Feed for unexpected overnight activity.
8. If anything needs attention, navigate to Alerts, Devices, or Tickets from the sidebar.

Practical Example: Client Has a Server Down

ACME Corp calls at 9am — their server is unresponsive.

1. Open the Dashboard Overview.
2. Look at the Device Health Map for any ACME device showing offline (grey dot).
3. Note the hostname of the offline device.
4. Click **Devices** in the sidebar.
5. Search for the ACME server. Check the **Last Seen** time.
6. Check **Alerts** to see if an "offline" alert fired.
7. Create a ticket to document the issue.
8. Contact ACME to say you are investigating.

> **TIP**
>
> The dashboard does not auto-refresh. Press F5 before making decisions based on the numbers you see.

Chapter 12: Devices — Monitoring Your Fleet

What it is

The Devices page shows every registered machine — every computer or server that has the agent installed and has checked in, plus any network-discovered agentless devices. This is the most information-dense page in the system.

Who uses it

All roles. Technicians and administrators interact with it most frequently.

OS Filter Tabs

The Devices page is divided into seven tabs, each showing a count badge:

Tab	What it shows
All	Every device — agent-managed and agentless
Windows	Agent-managed Windows computers
macOS	Agent-managed Mac computers
Linux	Agent-managed Linux machines
Android	Phones and tablets discovered via network scan, identified by Android OUI
iOS	Apple phones and tablets discovered via network scan, identified by Apple OUI
Agentless	All network-discovered devices regardless of type

Two Device Display Modes

Agent-managed devices (Windows/macOS/Linux tabs) show:

- Hostname, IP address, OS, last seen, online/offline status dot
- Live CPU, RAM, and disk usage gauges
- Remote reboot and shutdown buttons
- Customer assignment control

Agentless devices (Android/iOS/Agentless tabs) show:

- IP address, MAC address, vendor badge (e.g., "Apple, Inc."), platform badge
- Last seen timestamp, online/offline status dot
- **Ping Now** button — triggers an immediate reachability check and updates online status
- **Edit** button — opens an inline form to manually correct the hostname, platform (windows/mac/linux/android/ios/unknown), and device type (desktop/laptop/mobile/server/unknown). Useful when automatic detection could not identify the device (see Chapter 15).
- **Delete** button — removes the agentless device record permanently
- Customer assignment control

> **NOTE**
>
> Agentless devices do not report CPU, RAM, or disk metrics. They are presence-monitored only — the system confirms they are reachable on the network, nothing more.

What You See (Agent Devices)

Column	Description
Hostname	The device's computer name (e.g., ACME-SRV01)
IP Address	The device's local IP address
OS	Operating system (e.g., Windows 10, Windows Server 2022)
Status	Online (green dot) or Offline (grey dot)
Last Seen	Timestamp of the last heartbeat
CPU %	Current CPU usage percentage
RAM %	Current RAM usage percentage
Disk %	Primary disk usage percentage
Customer	Which customer this device belongs to

Understanding Online vs Offline

A device is **online** if its agent has sent a heartbeat in the last few minutes. If no heartbeat has arrived — because the machine is off, the agent crashed, or there is a network problem — the device shows **offline**.

> **IMPORTANT**
>
> An offline device is not necessarily broken. It could be powered off outside business hours. Always check the Last Seen timestamp to understand how long it has been offline.

Status Categories

Status	Meaning
Healthy	Online, all metrics in normal range
Warning	Online, at least one metric 75–90%
Critical	Online with a metric >90%, or has an active critical alert
Offline	Agent not reporting — no recent heartbeat

Step-by-step: Viewing a Device's Detailed Metrics

1. Click **Devices** in the sidebar.
2. Find the device. Use search or filter controls if available.
3. Click the device row or name to expand it.
4. You will see:
• Full OS name and version
• Platform (Windows, Linux, macOS)
• CPU, RAM, and disk metrics
• Last seen timestamp
• Agent registration date
• Uptime

Step-by-step: Investigating a Device After an Alert

You have received an alert that ACME-SRV01 has high CPU. Here is what to do:

1. Go to **Devices** in the sidebar.
2. Find ACME-SRV01.
3. Check the CPU column — is it still high?
4. Click the device to expand details.
5. Look at the CPU history if available. Is this a spike or sustained load?
6. Check Last Seen — is the device still online?
7. If CPU is still high and device is online, consider:
• Going to **Scripts** and running a "Get Running Processes" script.
• Going to **Maintenance** and scheduling a reboot if agreed with the customer.
8. Create or update a ticket with your findings.

Practical Example: New Technician Surveys Their Assigned Clients

James is new, assigned to three small business clients. First day:

1. Go to **Customers** in the sidebar. Note the three client names.
2. Go to **Devices**.
3. Look for devices with those customer names in the Customer column.
4. Note which are online, which are offline, any with high metrics.
5. Go to **Dashboard Overview** → Device Health Map for a visual summary.

> **TIP**
>
> If a device has not been seen for several hours during business hours, contact the customer proactively. Do not wait for them to call you.

Chapter 13: Alerts — Staying Ahead of Problems

What it is

The Alerts page manages all active system notifications. An alert is generated when a device's metric crosses a threshold you defined. Alerts are your early warning system.

The page has two tabs: **Active Alerts** and **Alert Rules**.

Who uses it

All roles can view alerts. Technicians and administrators can acknowledge, resolve, and create rules.

The Active Alerts Tab

Four stat cards at the top:

- **Open Alerts:** Total unresolved
- **Critical:** How many are critical severity
- **Acknowledged:** Seen but not yet resolved
- **Warning:** How many are warning severity

The severity filter lets you show only critical, warning, or info alerts.

Each alert row (when expanded) shows:

- A colored severity bar (red = critical, amber = warning, blue = info)
- Severity and status badges
- Which device triggered it
- The timestamp
- Full alert message
- Two action buttons: **Acknowledge** and **Resolve**

Alert Severity

Severity	Color	Meaning
critical	Red	Immediate action required — service may be impacted
warning	Amber	Attention needed soon — degraded performance
info	Blue	Informational — no immediate action needed

Alert Status

Status	Meaning
open	New alert, not yet seen
acknowledged	A technician has seen it and is dealing with it
resolved	The underlying issue has been fixed

Step-by-step: Acknowledging an Alert

Acknowledging means "I have seen this and I am dealing with it." It does not mean the problem is fixed.

1. Click **Alerts** in the sidebar.
2. Find the alert. Critical alerts appear first.
3. Click the alert to expand it.
4. Click **Acknowledge**.
5. Confirmation: "Alert acknowledged." The badge changes from OPEN to ACKNOWLEDGED.

Step-by-step: Resolving an Alert

Resolve only after actually fixing the underlying issue.

1. Find and expand the alert.
2. Verify the problem is fixed (check device metrics on the Devices page).
3. Click **Resolve**.
4. Confirmation: "Alert resolved." The alert is removed from the Active list.

The Alert Rules Tab

Without rules, no alerts are ever generated. Rules define when alerts are created.

Each rule shows:

- Rule name
- Metric monitored (cpu, ram, disk, battery, offline)
- Condition (`gt 90` means "greater than 90%")
- Cooldown period (minutes before the rule can fire again for the same device)
- Severity level
- Active/Inactive status and a toggle

Step-by-step: Creating an Alert Rule

1. Click **Alerts** → **Alert Rules** tab.
2. Scroll to the **Create Alert Rule** section.
3. Fill in the form:
- **Rule Name:** e.g., "High CPU Warning"
- **Severity:** critical, warning, or info
- **Metric:** cpu, ram, disk, battery, or offline
- **Operator:** `gt` (greater than), `gte` (≥), `lt` (less than), `lte` (≤)
- **Threshold (%):** The value to compare against
- **Cooldown (min):** How long before this rule can fire again for the same device
- **Auto-create ticket:** Check to auto-create a ticket every time this rule fires
4. Click **Create Rule**.
5. The rule will appear in the list and start evaluating on the next heartbeat.

Practical Example: Alert Fires at 2am

It is 2am. Alex (on-call) gets an email: "CRITICAL: Disk at 97% on CORP-SRV02."

1. Alex logs in from their laptop.
2. Goes to **Alerts** → finds the critical alert.
3. Clicks **Acknowledge** — marks it as seen.
4. Goes to **Devices** → finds CORP-SRV02 → checks disk details. Drive C: at 97.3%.
5. Goes to **Maintenance** → selects CORP-SRV02 → confirms checkbox → clicks **Delete Temp Files**.
6. Waits 5 minutes. Disk drops to 94% — still critical but immediate action taken.
7. Creates a ticket: "CORP-SRV02 critical disk usage — need full cleanup" with priority Critical.
8. Goes to **Scripts** → runs "Get Largest Files" on CORP-SRV02 → adds output to the ticket as a comment.
9. Resolves the alert.
10. In the morning, a senior technician completes the full cleanup and closes the ticket.

Recommended Baseline Alert Rules

Set up these rules in every new environment:

Rule Name	Metric	Operator	Threshold	Severity	Cooldown	Auto-ticket
Critical CPU	cpu	gt	90	critical	15	Yes
CPU Warning	cpu	gt	75	warning	30	No
Critical RAM	ram	gt	90	critical	15	Yes
RAM Warning	ram	gt	80	warning	30	No
Critical Disk	disk	gt	90	critical	60	Yes
Disk Warning	disk	gt	75	warning	120	No
Device Offline	offline	—	—	warning	60	No
Low Battery	battery	lt	20	warning	60	No

Chapter 14: App Center — Software Inventory

What it is

The App Center shows all software installed on a selected device. Use it to audit what is running on client machines, check software versions, identify unauthorized applications, and prepare for patch management.

Who uses it

IT support staff, technicians, and administrators.

What You See

After selecting a device:

- A summary showing how many software packages are installed
- A searchable, filterable table of every installed application with:
- Software name
- Version number
- Publisher

Step-by-step: Viewing Installed Software

1. Click **App Center** in the sidebar.
2. Use the device selector dropdown to choose a device.
3. A table loads with all software on that device.
4. Use the **Search** field to filter by application name, version, or publisher.

> **NOTE**
>
> Software inventory is collected every 6 hours. Data reflects the last scan. If software was installed in the past 6 hours, it may not appear yet.

Practical Example: Compliance Audit for Adobe Acrobat

Your manager asks you to confirm all DataSafe Ltd devices have Adobe Acrobat version 2024 or later.

1. Go to **Customers** → note all devices for DataSafe Ltd.
2. Go to **App Center**.
3. For each device, select it, then search for "Adobe Acrobat".
4. Note the version.
5. Create a ticket or note for any device not meeting the requirement.

Chapter 15: Network Discovery

What it is

Network Discovery performs a real ICMP ping sweep of a subnet you specify. It discovers all reachable hosts — computers, phones, printers, IoT devices — identifies their MAC address and vendor, and lets you save them permanently to your device fleet. Phones and tablets discovered this way appear in the Android and iOS tabs on the Devices page and are pinged automatically every 5 minutes.

Who uses it

Technicians and administrators.

Scan Results Table

Column	Description
Platform icon	Visual indicator (■ Windows, ■ Apple/iOS, ■ Linux, ■ Android, ■ Unknown)
IP Address	The discovered host's IP
MAC Address	Hardware MAC address from ARP table
Vendor	OUI lookup result (e.g., "Apple, Inc.", "Samsung Electronics")
Platform	Detected OS family
Status	Online (responded to ping) or Offline

Step-by-step: Running a Network Scan

1. Click **Network Discovery** in the sidebar (under TOOLS).
2. In the **Subnet** field, enter the network range in CIDR notation. Example: `192.168.1.0/24`
3. Optionally select a **Customer** to assign all discovered devices to.
4. Click **Scan Network**.
5. A spinner shows while the scan runs (typically 15–30 seconds for a /24 subnet).
6. The results table populates automatically when the scan completes.

> **WARNING**
>
> Running a network scan on a network with intrusion detection may trigger security alerts. Check with the client before scanning sensitive networks.

> **NOTE**
>
> The scan uses concurrent ICMP ping (50 parallel threads). On large subnets (/16 or wider), expect longer scan times and consider narrowing the range.

How Platform Detection Works

The system uses three stages to identify each discovered device's operating system, stopping as soon as a match is found:

1. **OUI vendor lookup** — the first 6 characters of the MAC address are matched against a built-in database of 500+ hardware manufacturers. Apple MACs → iOS, Samsung/Google/OnePlus MACs → Android, etc.
2. **Port probing** (fallback) — if OUI lookup cannot determine the platform, the system tries connecting to well-known ports:

• Port 62078 → iOS (iTunes Wi-Fi sync port)
• Port 5555 → Android (ADB debug port)

- **Two or more** of ports 445, 3389, 139 → Windows (requires 2 to avoid false-positives from routers and NAS devices that only expose SMB on port 445)
- Port 548 → macOS (Apple Filing Protocol)
- Port 22 → Linux (SSH)

Router and gateway devices (Fritz!Box, router, modem, etc.) are detected via their hostname and skipped entirely — they appear in the scan results for reference but are never added to your device fleet.

3. **Hostname keyword matching** (final fallback) — if both OUI and port probe fail, the system checks the device's reverse-DNS hostname (the name your router assigns, e.g. `Galaxy-S21-Ultra.fritz.box`) against 50+ keywords:

- Android brands: samsung, galaxy, pixel, xiaomi, redmi, poco, huawei, honor, oppo, vivo, realme, motorola, nokia, zte, and more
- Samsung model numbers: S10–S24, Note, A12–A73, Fold, Flip, Ultra
- iOS: iphone, ipad, ipod

Fritz!Box, ASUS, TP-Link, and most home routers assign the device's advertised name as the hostname, making this stage effective even when MAC randomization defeats OUI lookup and ADB is disabled.

> **NOTE**
>
> A device may still appear as "Unknown" if: (a) its MAC is randomized AND all listed ports are blocked AND its router-assigned hostname contains no recognisable keywords, or (b) it was offline during the latest scan. Re-running a scan when the device is connected will trigger all three detection stages again. Devices previously stored as "Unknown" are automatically upgraded to the correct platform on the next successful scan — no manual intervention needed. If a device genuinely cannot be auto-detected, use the **Edit** button on its row in the Devices page.

Step-by-step: Saving Discovered Devices

1. Review the scan results. Verify the detected platform icons look correct.
2. Click **Save All to Devices**.
3. A confirmation shows how many devices were created, updated, or skipped (skipped = already registered with an agent).
4. Go to **Devices** → **Agentless** tab to see all saved devices.
5. Phones appear in the **Android** or **iOS** tabs depending on their detected platform. If shown as Unknown, use the Edit button to correct it.

Automatic Online/Offline Monitoring

Once devices are saved, the system pings them automatically every 5 minutes via a background task. If a device does not respond for more than 10 minutes, its status is set to offline. No manual action is needed — the Devices page reflects current reachability automatically.

Past Scan History

When no scan is running, the page shows the last 5 completed scans with their subnet, host count, and timestamp. Click any scan to review its results without re-scanning.

Practical Example: Investigating an Unauthorized Device

A client suspects an unauthorized device on their network.

1. Go to **Network Discovery**.
2. Enter the client's subnet (e.g. `192.168.10.0/24`) and click **Scan Network**.
3. Compare the results against the known device list — look for unrecognized MAC addresses or vendors.
4. Any unfamiliar IP or vendor warrants investigation.
5. Save the results so the device appears in the Agentless tab for ongoing monitoring.

PART IV — MANAGEMENT

Chapter 16: Managing Tickets

What it is

The Tickets page is the helpdesk ticketing system. Every support request, incident, or task should have a ticket. Tickets allow you to track work, communicate with teammates, and maintain a complete history of everything done for each client.

Who uses it

IT support staff, technicians, and administrators. Viewers can see tickets but cannot create or update them.

Ticket Fields

Field	Description	Options
Title	Brief summary of the issue	Free text (required)
Description	Detailed explanation	Free text
Customer	Which client this relates to	Dropdown (required)
Priority	Urgency level	low, medium, high, critical
Status	Current state	open, in_progress, resolved, closed
Source	How it was created	manual, alert (auto-created), agent

Step-by-step: Creating a New Ticket

1. Click **Tickets** in the sidebar.
2. Click the **+ New Ticket** expander at the top of the page.
3. In **Title**, type a brief summary. Example: "Server offline — ACME Corp"
4. In **Priority**, select the urgency:
• **critical** — complete outage or data risk
• **high** — significant impact, not yet down
• **medium** — non-urgent issue
• **low** — request or minor issue
5. In **Description**, write full details: what the issue is, when it started, what has been tried.
6. In **Customer**, select the relevant customer. The customer must already exist in the system.
7. Click **Create Ticket**.
8. A green "Ticket created successfully!" message confirms success.

> **WARNING**
>
> The customer dropdown will show "— no customers —" if no customers exist yet. Create the customer in Chapter 17 first.

Step-by-step: Finding a Ticket

1. Click **Tickets** in the sidebar.
2. Use the filter bar:
• **Search field:** Type any word from the title or description.
• **Status dropdown:** Select open, in_progress, resolved, or closed. Select "All" for everything.
• **Priority dropdown:** Filter by urgency level.
3. The caption below filters shows how many tickets match.

Step-by-step: Updating a Ticket Status

1. Find and expand the ticket.
2. In the **Update Status** section, click the dropdown and select the new status:
• **open** — just created, not yet worked on
• **in_progress** — someone is actively working on it
• **resolved** — the issue has been fixed, awaiting confirmation
• **closed** — fully complete, no further action needed
3. Click **Update Status**.
4. A green confirmation appears and the badge changes color.

Step-by-step: Adding a Comment

1. Find and expand the ticket.
2. In the **Add Comment** section, type your comment.
3. Tick **Internal note** if the comment is for team eyes only.
4. Click **Post Comment**.

> **TIP**
>
> Use comments to maintain a running log of every action. Future teammates reading the ticket should be able to understand the entire history from comments alone.

Ticket Status Colors

• **open** — red (needs attention)
• **in_progress** — amber (being worked on)
• **resolved** — green (fix applied)
• **closed** — grey (done)

Practical Example: Client Calls With a Complaint

It is 2pm. TechCorp Ltd calls — their accounting software is very slow.

1. Go to **Tickets** → click **+ New Ticket**.
2. Title: "Accounting software slow performance — TechCorp Ltd"
3. Priority: **high**
4. Description: "Client reports QuickBooks running extremely slowly since this morning. All 5 workstations affected. No recent changes reported."
5. Customer: "TechCorp Ltd"
6. Click **Create Ticket**. Note the ticket ID.
7. Go to **Devices** → find TechCorp Ltd's server → check CPU.
8. CPU is at 98% — return to the ticket, expand it, add a comment: "Checked server TECHCORP-SRV01 — CPU at 98%. Investigating."
9. Update status to **in_progress**.
10. Resolve the issue. Add a comment explaining what was done.
11. Update to **resolved** and confirm with client.
12. After confirmation, update to **closed**.

Chapter 17: Working with Customers

What it is

The Customers page manages the client organizations that own the devices you support. Every device belongs to a customer. Every ticket must be associated with a customer.

Who uses it

IT support staff, technicians, and administrators.

Customer Fields

Field	Description	Options
Name	Company or client name	Free text (required)
Email	Primary contact email	Email format
Phone	Contact phone number	Free text
Tier	Support level	standard, premium, enterprise
Primary Technician	Assigned team member	Selected from users

Step-by-step: Creating a New Customer

1. Click **Customers** in the sidebar.
2. Click **+ New Customer**.
3. Enter the **Name** — this is required and will identify the customer system-wide.
4. Fill in **Email**, **Phone**.
5. Select the appropriate **Tier**:

- **standard** — basic support, standard response times
- **premium** — priority support, faster response
- **enterprise** — highest tier, dedicated support

6. Optionally assign a **Primary Technician**.
7. Click **Save**.
8. The customer now appears in all dropdowns throughout the system.

Practical Example: Onboarding a New Client

You just signed Greenway Manufacturing Ltd.

1. Go to **Customers** → click **+ New Customer**.
2. Name: "Greenway Manufacturing Ltd"
3. Email: `it-contact@greenway.com`
4. Phone: `+1 555 234 5678`
5. Tier: **premium** (they signed a premium support contract)
6. Primary Technician: assign yourself or the dedicated technician.
7. Click **Save**.
8. Deploy the agent on Greenway's machines (Chapter 6).
9. Within minutes of the agent connecting, the devices appear under Greenway Manufacturing Ltd on the Devices page.

> **TIP**
>
> The tier affects billing calculations on the Billing page — set it correctly during onboarding.

Chapter 18: Automation Profiles

What it is

Automation Profiles define bundles of maintenance tasks that run automatically on a schedule — daily, weekly, monthly, or on demand. A single profile can combine OS patching, software patching, disk maintenance, and cleanup tasks, and run across all your managed devices with no manual intervention.

Who uses it

Administrators and senior technicians.

Profile Structure

An automation profile contains:

- **Name and status:** Identifier and whether active or inactive.
- **Schedule:** When to run — daily, weekly, monthly, or once. Plus day of week and time.
- **Notification emails:** Email addresses to notify after the profile runs.
- **Run on newly installed agents:** Whether new devices automatically get this profile.

- **Four task columns:**

Column	Tasks
OS Patch Management	Which Windows update categories to install
Software Patch Management	Which software to update, which to exclude
Disk Management	Defragment, Check Disk
Maintenance	Restore Point, Temp Files, Browser History, Reboot, Shutdown

The Profile List Tab

Shows all profiles as cards. Each card shows:

- Status dot (green = active, grey = inactive)
- Profile name, badge, schedule type, last run time
- A **Run Now** button — immediately queues the profile on all assigned devices

Step-by-step: Creating an Automation Profile

1. Click **Automation** in the sidebar.
2. Click the **Create / Edit Profile** tab.
3. Leave the dropdown on **— New Profile —**.
4. Enter a **Profile Name**. Example: "Weekly Maintenance — Standard Clients"
5. Check **Active** to enable.
6. Set **Schedule** to `weekly`, Day to `sunday`, Time to `02:00`.
7. Enter notification email(s), comma-separated.
8. Under **OS Patch Management**: check Critical updates and Security updates.
9. Under **Disk Management**: check Run Checkdisk. Do not check Defragment unless you are sure these are HDDs.
10. Under **Maintenance**: check Create System Restore Point and Delete Temp Files.
11. Click **Save Profile**.
12. A "Profile saved!" confirmation appears.

Step-by-step: Running a Profile Immediately

1. Go to **Automation** → **Profile List** tab.
2. Find the profile.
3. Click **Run Now**.
4. A confirmation shows how many devices the profile was queued on.
5. Navigate to **Maintenance** → Recent Maintenance Runs to watch tasks complete.

Practical Example: Onboarding a New Client with Weekly Patching

Greenway Manufacturing needs weekly security patches every Sunday night.

1. Go to **Automation** → **Create / Edit Profile**.

2. Name: "Greenway Manufacturing — Weekly Security Patches"
3. Active: Checked. Schedule: weekly, Sunday, 23:00.
4. Notification: `it-contact@greenway.com`, your email.
5. OS Patches: Critical + Security updates only.
6. Maintenance: Create Restore Point + Delete Temp Files.
7. Save.
8. On Monday morning, check the **Maintenance** run log to verify patches were applied.

> **WARNING**
>
> Enabling Reboot in a profile is a significant action. Always confirm with clients that a reboot is acceptable before enabling it.

PART V — PATCHING

Chapter 19: OS Patch Management

What it is

The OS Patches page manages Windows Update deployment across all managed devices. You can see which patches are pending approval, approve them in batches, review patch history, and configure policies.

Who uses it

Technicians and administrators.

The Four Stat Cards

Card	Description
Pending	Patches waiting for approval before deployment
Approved	Patches approved but not yet deployed
Deployed	Patches successfully deployed
Compliance %	Percentage of devices that are fully patched

Compliance color coding: Green (90%+) = good, Amber (70–89%) = needs attention, Red (<70%) = poor.

The Pending Patches Tab

Shows all patches waiting for your approval. Each patch shows:

- Patch name (including KB number)
- Type badge: critical, security, definition, rollup, feature, driver, update
- Which device reported this patch as available

To approve patches:

1. Use the checkboxes to select patches you want to approve.
2. Select one, several, or all.
3. Click **Approve Selected (N)**.
4. Approved patches are queued for deployment on next agent check-in.

> **NOTE**
>
> Approving a patch does not immediately install it. The agent receives the approval on its next heartbeat cycle and proceeds with installation.

The Patch History Tab

Shows all patches across all statuses:

Column	Description
Patch Name	Full descriptive name
Type	Color-coded patch type badge
Status	DEPLOYED, APPROVED, PENDING, or FAILED
Device	Which device this patch applies to
Date	Deployment or creation date

Step-by-step: Approving This Week's Security Patches

1. Go to **OS Patches** → **Pending Patches** tab.
2. Prioritize patches with red "critical" or orange "security" badges.
3. Check those patches.
4. Click **Approve Selected (N)**.
5. Success message confirms how many were approved.
6. Check back in a few hours — switch to **Patch History** and verify they show as DEPLOYED.

Practical Example: Checking Patch Compliance for a Client

DataSafe Ltd asks you to confirm all their servers have the latest security patches.

1. Go to **OS Patches** → **Patch History** tab.
2. Find all entries for DataSafe devices in the Device column.
3. Verify all show status DEPLOYED.
4. Check the Compliance % stat card.
5. If below 90%, go to Pending Patches and approve remaining patches.
6. Generate a `patch_summary` report (Chapter 24) for DataSafe Ltd as documentation.

Chapter 20: Software Patches

What it is

The Software Patches page manages third-party software updates on individual devices — applications like Chrome, Firefox, 7-Zip, VLC, and others that can be updated via winget or chocolatey package managers.

Who uses it

Technicians and administrators.

Page Layout

Left column:

- Device selector (only online devices shown)
- Device info card (hostname, OS, IP)
- **Check for Updates** button

Right column:

- Searchable table of all installed software packages

What the Software List Shows

The right column displays all software installed on the selected device, collected by the agent from two sources:

- **Windows Registry** — the most comprehensive source; covers everything in Add/Remove Programs
- **winget** — Microsoft's package manager; shows packages it knows about with their IDs

Each row shows: **Name**, **Version**, and **Publisher** (where available). Use the search box to filter by name or publisher.

> **NOTE**
>
> The **Check for Updates** button is a placeholder for a future winget/Chocolatey update engine (Phase 6 roadmap). It does not currently trigger updates — use Automation Profiles (Chapter 18) for bulk software patching.

Step-by-step: Finding a Specific Application

1. Click **Software Patches** in the sidebar.
2. Select an online device from the left column dropdown.
3. The right column loads the software list automatically.
4. Type the application name or publisher in the **Search** box to filter.

> **TIP**
>
> The **Installed** counter (top right of the software list) shows the total number of packages found on the selected device.

> **NOTE**
>
> For bulk software patching across many devices, use Automation Profiles (Chapter 18) — software patching here is device-by-device.

PART VI — TOOLS

Chapter 21: Scripts — Running Custom Automation

What it is

The Scripts page is one of the most powerful features in the system. It lets you run automated scripts on managed devices from a central interface — no remote desktop needed. You can run built-in scripts from the library, upload custom scripts, and review execution history.

Who uses it

Technicians and administrators for day-to-day use. Developers for creating and maintaining the script library.

Supported Script Types

Type	Badge	Language	Typical Use
ps1	Blue	PowerShell	Windows automation, registry, services, users
bat	Orange	Windows Batch	Simple commands, legacy compatibility
py	Green	Python	Complex logic, API calls, data processing
sh	Purple	Shell/Bash	Linux/macOS agents

The Three Tabs

Library Tab Browse all scripts. Each script shows as an expandable card:

- Tag: ■ Built-in or ■ Custom
- On expansion: type badge, OS target, creation date, description
- Device multi-select (only online devices)
- Timeout setting (10–900 seconds, default 300)
- **Run** button

Upload Tab Create a new script:

- Script Name, Type, Description, and Script Content
- Click **Upload Script** to save to the library

Run History Tab Shows last 50 executions:

- Status icon: ■ success, ■ failed, ■ queued, ■ running, ■ timeout
- Script name, device hostname, status, triggered timestamp

Expanding a run entry shows duration, exit code, stdout, and stderr.

Step-by-step: Running a Script on a Single Device

1. Click **Scripts** in the sidebar.
2. In the **Library** tab, search for the script.
3. Click to expand it.
4. In the device multi-select, select the target device.
5. Adjust the **Timeout** if needed.
6. Click **Run**.
7. Confirmation: "Queued on 1 device(s)."
8. Go to **Run History**. The run appears as ■ QUEUED.
9. After ~60 seconds (next agent heartbeat), refresh — status will be ■ SUCCESS or ■ FAILED.
10. Expand the entry to see output.

Step-by-step: Running a Script on Multiple Devices

1. Follow steps 1–4 above.
2. In the multi-select, click multiple device names.
3. Click **Run**.
4. "Queued on [N] device(s)" confirms the count.
5. In Run History, one entry appears per device.

Step-by-step: Uploading a New Script

1. Click **Scripts** → **Upload** tab.
2. Enter a clear descriptive **Name**: "Get Top 10 Largest Files"
3. Select **Type**: `ps1`
4. In **Description**: "Lists the 10 largest files on Drive C: by size."
5. Paste or write your script code in the content area.
6. Click **Upload Script**.
7. Switch to **Library** tab and search to confirm it appears.

Reading Script Output

1. Go to **Run History** tab.
2. Find and expand the run.
3. **stdout** — normal output. This is your result data.
4. **stderr** — error output. Investigate even if exit code is 0.

A successful script: exit code 0, output in stdout.

Practical Example: Cleanup Script on 10 Devices

Your manager asks you to run the "Clear Temp Files" script on all 10 QuickPrint Co devices overnight.

1. Go to **Scripts** → **Library** → search for "temp" or "cleanup".
2. Find and expand the appropriate script.

3. In the device multi-select, select all 10 QuickPrint Co devices (confirm they are online on Devices page first).
4. Set timeout to 120 seconds.
5. Click **Run**.
6. Confirm: "Queued on 10 device(s)."
7. Check **Run History** after 5 minutes. All should show ■ SUCCESS.
8. If any show ■ FAILED, expand that entry and read stderr to diagnose.

Chapter 22: Disk Management

What it is

Disk Management gives a visual and actionable view of disk usage across any selected device. It shows gauge charts for each drive and provides action buttons to perform disk maintenance remotely.

Who uses it

Technicians and administrators.

What You See

After selecting a device:

Gauge Charts (up to 4 drives) Each drive gets a gauge chart showing percentage used:

- **Green:** Under 75% (healthy)
- **Yellow/Amber:** 75–90% (warning)
- **Red:** Over 90% (critical)

Summary Table Lists all disks with used space, total capacity, and percentage.

Action Buttons

Button	Effect	Risk
Defragment	Schedules disk defragmentation	LOW (do NOT use on SSDs)
Check Disk	Schedules chkdsk for next reboot	LOW
Clean Temp Files	Deletes temp files to free space	LOW

Step-by-step: Investigating High Disk Usage

1. Click **Disk Management** in the sidebar.
2. Select the target device.
3. View gauge charts. Red = needs attention.
4. Read the summary table for exact GB values.

5. If critically low: a. Click **Clean Temp Files** first. b. If still critical, go to **Scripts** and run "Get Largest Files". c. For HDDs only — consider Defragment.
6. Wait for the agent to complete the action.
7. Refresh the page to see updated disk metrics.

Practical Example: Emergency Disk Cleanup

Alert fires at 3pm: RETAIL-PC05 disk at 93%. Machine cannot be rebooted during business hours.

1. Go to **Disk Management** → select RETAIL-PC05.
2. Drive C: is at 93% (deep red).
3. Click **Clean Temp Files**. Queued message appears.
4. Wait 2 minutes. Go to **Devices** → check disk metric for RETAIL-PC05.
5. Disk drops to 88% (yellow zone). Immediate crisis managed.
6. Create a ticket for a full disk audit during the next maintenance window.

> **WARNING**
>
> Do not run Defragment on SSDs. It wears out SSD cells and provides no benefit.

Chapter 23: Maintenance Actions

What it is

The Maintenance page lets you perform remote actions on an online device: reboot, shutdown, create restore points, delete temp files, clear browser history, and schedule a disk check. It also shows recent maintenance run history.

Who uses it

Technicians and administrators.

Critical Safety Features

Only online devices appear — the selector lists only currently online devices.

Confirmation checkbox required — you must check "I confirm this action on the selected device" before any button works. Without it, you get a warning and nothing executes.

The Device Info Card

After selecting a device, a card shows:

- Hostname and IP (with green online dot)
- Operating System
- Last Seen timestamp
- Uptime

Verify this is the correct machine before taking any action.

Available Actions

Button	Effect	Risk Level
Reboot	Immediate reboot command	HIGH — interrupts active users
Shutdown	Immediate shutdown	HIGH — takes device offline
Create Restore Point	Windows system restore point	LOW — no disruption
Delete Temp Files	Removes temp files	LOW — safe
Clear Browser History	Clears browser saved data	MEDIUM — may affect user sessions
Check Disk	Schedules chkdsk at next reboot	LOW — no live disk changes

> **IMPORTANT**
>
> All six actions are fully functional. Reboot and Shutdown execute immediately when the agent receives the command (within 60 seconds). Always confirm with the client before rebooting or shutting down any device.

Step-by-step: Rebooting a Device

1. Click **Maintenance** in the sidebar.
2. Select the target device. Only online devices appear.
3. Verify the device info card shows the correct machine.
4. Before proceeding — verify the device is not actively in use and confirm with the customer.
5. Check the **"I confirm this action on the selected device"** checkbox.
6. Click **Reboot**.
7. "Reboot command sent to [hostname]." The command has been dispatched.
8. The device will appear offline for a few minutes.
9. After reboot, the agent reconnects. Verify the device returns to online on the Devices page within 10 minutes.

Step-by-step: Checking Recent Maintenance Runs

1. Scroll to the bottom of the Maintenance page.
2. The **Recent Maintenance Runs** table shows the last 20 runs.
3. Columns: Profile name, Device, Started, Finished, Status badge.
4. Status colors:

- **success** (green) — completed without errors
- **failed** (red) — error encountered
- **running** (amber) — currently in progress
- **pending** (grey) — queued, not yet started

Practical Example: Post-Patch Reboot

A patch was applied to CORP-SRV01. A reboot is required. It is 8pm (after hours).

1. Go to **Maintenance** → select CORP-SRV01.
2. Check Uptime in the device info card — 3 days (expected).
3. Tick "I confirm this action on the selected device."
4. Click **Reboot**.
5. Go to **Devices** → watch CORP-SRV01 go offline then return online.
6. Once back (5–10 minutes), verify the patch applied via **OS Patches** → Patch History.
7. Add a comment to the ticket: "CORP-SRV01 rebooted at 8:02pm. Back online at 8:09pm. Patch verified."

PART VII — BUSINESS

Chapter 24: Reports

What it is

The Reports page generates formal reports about your managed environment. Reports are useful for client meetings, monthly reviews, compliance audits, and billing discussions.

Who uses it

Administrators and senior technicians. The management team uses reports to present RMM service value to clients.

Report Templates

Template	What it covers
device_summary	All devices, status, OS versions, last seen times
patch_summary	Patch compliance, pending patches, deployed patches
alert_summary	All alerts in the date range, by severity and device
billing_summary	Billing totals, invoices generated, amounts due

Step-by-step: Generating a Report

1. Click **Reports** in the sidebar (under BUSINESS).
2. Click the **Generate** tab.
3. Select a **Template**.
4. Select the **Customer** (or All, if available).
5. Set the **Date Range** using Start Date and End Date.
6. Click **Generate**.
7. The report is created and available for download as PDF or Excel.

Step-by-step: Viewing Report History

1. Click **Reports** in the sidebar.
2. Click the **History** tab.
3. Each entry shows: report type, customer, date range, generated timestamp, and a **Download** button.

Practical Example: Monthly Client Report Package

At the end of each month, you send each client a summary of device health and incidents.

1. Go to **Reports** → **Generate** tab.
2. Template: `device_summary`, Customer: ACME Corp, Date range: entire previous month. Generate. Download.
3. Repeat for `alert_summary` and `patch_summary`.
4. Email the three reports to the ACME Corp contact.
5. File the reports in the History tab for future reference.

Tips

- If no data exists for the selected date range, the report will be empty or show "No data". Verify agents were active during that period.
- Reports are stored in the system and can be re-downloaded at any time from the History tab.

Chapter 25: Billing

What it is

The Billing page allows administrators to generate invoices for clients based on the number of managed devices during a billing period. It tracks invoice status: draft, sent, or paid.

Who uses it

Administrators. This is the financial management section of the system.

Billing Fields

Field	Description
Customer	The client being billed
Period Start	Start date of the billing period
Period End	End date of the billing period
Device Count	Number of managed devices in that period
Per-Device Rate	Monthly rate per device

Step-by-step: Generating an Invoice

1. Click **Billing** in the sidebar.
2. Find the invoice creation form.
3. Select the **Customer**.
4. Set **Period Start** and **Period End** dates.
5. Enter the **Device Count** (verify on the Devices page by filtering by customer).
6. Enter the **Per-Device Rate** (e.g., 25.00 for $25 per device per month).
7. Click **Generate Invoice**.
8. The invoice appears in the list with status "draft".

Invoice Status

Status	Meaning
draft	Created, not yet sent
sent	Delivered to the client
paid	Client has paid

Billing Summary Metrics

At the top of the Billing page:

- **Total Invoiced:** Sum of all invoice amounts
- **Paid:** Sum of "paid" invoices
- **Outstanding:** Total invoiced minus paid — what clients still owe

Step-by-step: End-of-Month Billing Run

1. Go to **Billing** at the end of every month.
2. Check device count for each customer (verify on Devices page).
3. Create an invoice for each customer using their contracted rate.
4. Note the invoice amounts.
5. Email invoices to each client.
6. Mark each invoice as "sent".
7. When payment is received, mark as "paid".
8. The Outstanding metric decreases automatically.

> **NOTE**
>
> Device count is entered manually. Always verify the actual number before billing to avoid errors.

Chapter 26: Administration

What it is

The Admin page is restricted to admin role users only. It provides three tabs: System Info, Audit Log, and Users. This is where you check system health, review all user actions, and manage user accounts.

Who uses it

Administrators only. Technicians and viewers are blocked from this page.

Tab 1: System Info

Four cards:

Current User: Your name, email, and role badge.

System: Configured API URL, dashboard URL, and database connection address.

Services: Live probe of service status:

- **Flask API** — probes `/api/health`. Shows green "Online" or red "Unreachable".
- **Streamlit Dashboard** — always shows Running (since you are using it).
- **PostgreSQL** — shows configured connection address.
- **Redis / Celery** — shows configured connection address.

> **NOTE**
>
> PostgreSQL and Redis show the configured address, not a live probe. To verify database connectivity, check the Flask API health endpoint.

Agent Enrollment Token: The `ORG_REGISTRATION_TOKEN` used to enrol managed machines.

- Token is masked by default (`a8b6ea••••••••••`).
- Click **Reveal** to show the full token, **Hide** to mask it again.
- Copy the revealed token and paste it into `config.ini → org_token` on any machine you want to enrol.
- Keep this token secret — anyone who has it can register devices to your organisation.
- After all agents are enrolled, consider rotating the token in `.env` and restarting the API.

Tab 2: Audit Log

Records every state-changing action in the system:

Action Type	Color	Examples
CREATE	Green	Ticket created, device registered, user created
UPDATE	Blue	Ticket status changed, device updated
DELETE	Red	Rule deleted, device removed
LOGIN	Purple	User logged in
LOGOUT	Amber	User logged out

Each entry shows: action type, resource type and ID, timestamp, IP address, and user email.

Filtering: Use the Action type dropdown and date range pickers to narrow results.

Step-by-step: Investigating a Suspicious Action

1. Go to **Admin** → **Audit Log** tab.
2. Set **Action type** filter to "DELETE".
3. Set a date range around when the suspected action occurred.
4. Look for unexpected DELETE events.

5. Note the User email and IP address.

Tab 3: Users

Shows a table of all registered users. Columns: Full Name, Email, Role, Created Date.

Available actions in the Users tab:

- **Create** new users (name, email, password, role) — includes **"Require password change on first login"** checkbox (checked by default). When enabled, the user must set a new password before accessing the dashboard.
- **Edit** existing users (change role, update name/email) — includes **"Require password change on next login"** checkbox to force a reset on an existing account.
- **Deactivate** users to block login without deleting records
- **Reset passwords** by editing and setting a new password

Step-by-step: Deactivating a Departed Employee

When someone leaves, deactivate their account immediately.

1. Go to **Admin** → **Users** tab.
2. Find the employee.
3. Click **Edit** or expand their row.
4. Click **Deactivate** and save.
5. Verify: check the Audit Log for any LOGIN events from that email after the deactivation date.

> **WARNING**
>
> JWT tokens have an expiry time. After deactivating an account, the user may retain access until their current token expires. For immediate lockout, rotate the `JWT_SECRET_KEY` in the `.env` file and restart the API.

Step-by-step: Monthly Admin Security Review

On the first Monday of each month:

1. Go to **Admin** → **Audit Log**.
2. Filter by the previous month's date range.
3. Look for:
- Unexpected DELETE actions
- LOGIN attempts from unusual IP addresses
- Unusual patterns of failed actions
4. Go to **Users** tab. Verify all listed users are current employees.
5. Go to **System Info** → Services card. Verify all services are healthy.

The Superadmin Account

What it is: A permanently present, highest-privilege account built into the system. It exists so that if all regular admin accounts are locked out, deactivated, or forgotten, the system can still be accessed and

recovered.

Key facts:

- There is exactly one superadmin account per system.
- It is automatically re-created every time the Flask API starts, if it does not already exist.
- It cannot be edited or deleted through the web interface. The Edit and Delete buttons are replaced by "Protected — use CLI" in the Users tab.
- It has a purple role badge in the sidebar.
- It has full access to every feature in the system.

Default credentials (change these immediately after installation):

- Email: `superadmin@rmm.local`
- Password: `SuperAdmin@RMM1`

Changing the superadmin email or password via environment variables:

Edit the `.env` file on the server and set:

```
SUPERADMIN_EMAIL=your-preferred-email@company.com
SUPERADMIN_PASSWORD=YourNewStrongPassword123
```

Then restart the Flask API. The account will be updated on next startup.

Emergency password reset (when locked out of the web interface):

If you cannot log in as superadmin and need to reset the password without a web session:

1. Open a terminal on the RMM server.
2. Navigate to the `api` folder:

```
Set-Location C:\RMM\RemoteManagementSystem\api
.\venv\Scripts\Activate.ps1
```

3. Run the reset tool:

```
python reset_superadmin.py YourNewPassword123
```

The password must be at least 10 characters. The tool confirms success and the new password takes effect immediately — no restart required.

> **WARNING**
>
> Keep the superadmin password in a secure password manager or sealed physical document. It is your last line of defence if all other admin access is lost.

> **NOTE**
>
> The superadmin account does not appear in the Audit Log's normal user activity in the same way as regular users — but its LOGIN events are still recorded.

PART VIII — TECHNICAL REFERENCE

Chapter 27: System Architecture

Component Map

```
┌─────────────────────────────────────────────────────────┐
│ Browser (http://localhost:8501)                         │
│ Streamlit Dashboard — Python/Streamlit                  │
│ dashboard/app.py + dashboard/pages/*.py                 │
└──────────────────────────┬──────────────────────────────┘
                           │ HTTP REST API calls
                           │ (JWT Bearer token in headers)
┌──────────────────────────▼──────────────────────────────┐
│ Flask API (http://localhost:5000)                       │
│ api/app.py, api/routes/*.py                             │
│ api/models/*.py (SQLAlchemy ORM)                        │
│ api/services/*.py (business logic)                      │
│ api/tasks/*.py (Celery async tasks)                     │
└──────────┬──────────────┬──────────────┬────────────────┘
           │              │              │
PostgreSQL Redis/Celery Agent API
localhost:5432 localhost:6379 /api/agent/*
db: rmmdb Task queue
           │
┌──────────▼──────────────┐
│ Celery Workers          │
│ - Alert evaluation      │
│ - Patch deployment      │
│ - Script dispatch       │
└─────────────────────────┘
           │
┌──────────▼──────────────┐
│ Python Agent            │
│ agent/rmm_agent.py      │
│ Heartbeat: 60s          │
│ SW scan: 6h             │
└─────────────────────────┘
```

Service Startup Order

1. **PostgreSQL** — Windows Service, starts automatically
2. **Redis/Memurai** — Windows Service, starts automatically
3. **Flask API** — `cd api ; python app.py`
4. **Celery Worker** — `cd api ; celery -A tasks.celery_app worker --pool=solo -l info`
5. **Celery Beat** — `cd api ; celery -A tasks.celery_app beat -l info`
6. **Streamlit Dashboard** — `cd dashboard ; streamlit run app.py`
7. **Agent (on managed machines)** — `cd agent ; python rmm_agent.py` (run as Administrator)

Directory Structure

```
RemoteManagementSystem/
■■■ api/
■ ■■■ app.py # Flask application factory
■ ■■■ config.py # Configuration (pool_size=10, max_overflow=20)
■ ■■■ extensions.py # SQLAlchemy, JWT, Celery setup
■ ■■■ models/ # SQLAlchemy model files
■ ■■■ routes/ # API route blueprints
■ ■■■ tasks/ # Celery task modules
■ ■■■ utils/
■ ■ ■■■ builtin_scripts.py # 7 built-in PS1 scripts
■ ■ ■■■ notifications.py # SMTP email sender
■ ■■■ reports/ # CSV report output directory
■ ■■■ seed.py # DB seed: admin user + default data
■■■ agent/
■ ■■■ rmm_agent.py # Main loop: register/heartbeat/tasks
■ ■■■ collector.py # Metrics, software inventory, patch scan
■ ■■■ heartbeat.py # API client
■ ■■■ executor.py # Task execution engine
■ ■■■ script_runner.py # Script execution
■ ■■■ config.ini # Agent configuration
■■■ dashboard/
■ ■■■ app.py # Login page
■ ■■■ pages/ # 16 Streamlit page files
■ ■■■ utils/
■ ■■■ auth.py # JWT login/logout/session management
■ ■■■ api_client.py # RMMClient: session reuse, retry, refresh
■ ■■■ nav.py # Shared sidebar component
■ ■■■ styles.py # CSS injection, stat cards, badges
■ ■■■ formatters.py # Date, byte, color formatting
■■■ scripts_library/ # Script templates on disk
```

Authentication Flow

1. User submits email + password to the login form.
2. `utils/auth.py` calls `POST /api/auth/login`.
3. API validates credentials using bcrypt.
4. If valid, API returns a JWT access token and refresh token.
5. Dashboard stores the token in `st.session_state["access_token"]`.
6. All 16 pages call `require_auth()` at load time.
7. All API calls include the token as `Authorization: Bearer <token>`.
8. On 401, the dashboard auto-calls `POST /api/auth/refresh` and retries once.

Agent Registration Flow

1. Agent reads `config.ini` on startup.
2. If `device_id` is blank, calls `POST /api/agent/register` with hardware info and `org_token`.
3. API creates a device record and returns `device_id` and `agent_token`.
4. Agent saves these to `agent_state.json`.
5. Every 60 seconds, agent calls `POST /api/agents/<device_id>/heartbeat` with current metrics.
6. Celery evaluates alert rules against new metrics every 60 seconds.
7. Agent polls `GET /api/agents/<device_id>/tasks` for queued commands.

8. Every 6 hours, agent scans for Windows updates and reports via `PUT /api/agents/<device_id>/patches`.

Database Models Overview

Model	Table	Key Fields
User	users	id, email, password_hash, role, is_active
Customer	customers	id, name, slug, email, phone, tier
Device	devices	id, hostname, os_name, customer_id, is_online, last_seen
DeviceMetrics	device_metrics	id, device_id, cpu_pct, ram_pct, disk_pct
InstalledSoftware	installed_software	id, device_id, name, version, publisher
AlertRule	alert_rules	id, name, metric, operator, threshold, severity, cooldown_minutes
Alert	alerts	id, rule_id, device_id, severity, message, status
Ticket	tickets	id, title, description, customer_id, priority, status
TicketComment	ticket_comments	id, ticket_id, author_id, body, is_internal
Script	scripts	id, name, file_type, content, is_builtin
ScriptRun	script_runs	id, script_id, device_id, status, exit_code, stdout, stderr
PatchRecord	patch_records	id, device_id, patch_name, kb_id, status
AutomationProfile	automation_profiles	id, name, schedule_type, is_active
Report	reports	id, name, template_type, customer_id, file_path
Invoice	invoices	id, customer_id, period_start, period_end, device_count, total, status
AuditLog	audit_logs	id, user_id, action, resource_type, ip_address

Chapter 28: Script Writing Guide

Overview

Scripts are the automation workhorses of the RMM. Write them in PowerShell (ps1), Windows Batch (bat), Python (py), or Shell (sh). The agent executes them and captures stdout and stderr, which are sent back to the RMM and displayed in Run History.

PowerShell (ps1) — Recommended for Windows

Basic template:

```
# Script: Get System Information
# Description: Returns OS, CPU, RAM, and disk info

try {
$os = Get-WmiObject Win32_OperatingSystem
$cpu = Get-WmiObject Win32_Processor
$disk = Get-WmiObject Win32_LogicalDisk -Filter "DriveType=3"

Write-Output "=== System Information ==="
Write-Output "OS: $($os.Caption) Build $($os.BuildNumber)"
Write-Output "CPU: $($cpu.Name)"
Write-Output "RAM: $([math]::Round($os.TotalVisibleMemorySize / 1MB, 2)) GB total"

Write-Output "=== Disk Usage ==="
foreach ($d in $disk) {
$pct = [math]::Round((1 - ($d.FreeSpace / $d.Size)) * 100, 1)
Write-Output "Drive $($d.DeviceID): $pct% used ($([math]::Round($d.FreeSpace/1GB,1)) GB
free)"
}
exit 0
} catch {
Write-Error "Script failed: $_"
exit 1
}
```

Key PS1 conventions:

- Use `Write-Output` (not `Write-Host`) — `Write-Host` goes to console, not stdout.
- Use `Write-Error` for error messages — appears in stderr.
- Exit with `exit 0` on success, `exit 1` on failure.
- Wrap in try/catch blocks.

Example: Get 10 Largest Files:

```
try {
$files = Get-ChildItem -Path "C:\" -Recurse -File -ErrorAction SilentlyContinue |
Sort-Object Length -Descending |
Select-Object -First 10

Write-Output "Top 10 largest files on C:"
Write-Output "----------------------------------------"
foreach ($f in $files) {
$sizeMB = [math]::Round($f.Length / 1MB, 1)
Write-Output "$sizeMB MB $($f.FullName)"
}
exit 0
} catch {
Write-Error "Error: $_"
exit 1
}
```

Windows Batch (bat)

```
@echo off
:: Script: List Running Services
net start
if %ERRORLEVEL% EQU 0 (
echo Services listed successfully.
exit /b 0
) else (
echo Error. Code: %ERRORLEVEL%
exit /b 1
)
```

Python (py)

```
#!/usr/bin/env python3
import sys
import subprocess

def main():
try:
result = subprocess.run(
["powershell", "-Command",
"Get-Process | Sort-Object CPU -Descending | Select-Object -First 10 | Format-Table
Name,CPU -AutoSize"],
capture_output=True, text=True, timeout=30
)
if result.returncode == 0:
print(result.stdout)
return 0
else:
print(f"Error: {result.stderr}", file=sys.stderr)
return 1
except Exception as e:
print(f"Script error: {e}", file=sys.stderr)
return 1

if __name__ == "__main__":
sys.exit(main())
```

Best Practices for All Scripts

1. **Always include a description** — future team members will need it.
2. **Use exit codes correctly** — 0 = success, non-zero = failure.
3. **Handle errors** — never let scripts fail silently.
4. **Limit output** — use `Select-Object -First 50` to avoid filling the database.
5. **Test on a dev device** before running on client machines.
6. **Never hardcode credentials** — no passwords or API keys in script content.
7. **Idempotent where possible** — scripts that can run multiple times safely are safer.

Chapter 29: Alert Rule Design

Rule Anatomy

```
Rule: "Critical CPU Alert"
■■■ Metric: cpu
■■■ Operator: gt (greater than)
■■■ Threshold: 90 (%)
■■■ Severity: critical
■■■ Cooldown: 15 (minutes)
■■■ Auto-ticket: true
```

When any device's CPU exceeds 90%, a critical alert is created. It cannot fire again for the same device for 15 minutes.

Available Metrics

Metric	Measures	Threshold Unit
cpu	CPU usage from latest heartbeat	% (0–100)
ram	RAM usage from latest heartbeat	% (0–100)
disk	Primary disk usage	% (0–100)
battery	Battery level (laptops)	% (0–100)
offline	Device stops sending heartbeats	N/A

Operators

Operator	Symbol	Use When
gt	>	Alert when metric goes above a level
gte	>=	Alert when metric reaches or exceeds a level
lt	<	Alert when metric drops below (low battery)
lte	<=	Alert when metric falls to or below a level

Cooldown Tuning

The cooldown prevents alert flooding. A 5-minute cooldown on a device constantly at 95% CPU means 12 alerts per hour for that device alone.

Severity	Recommended Cooldown
critical	15–30 minutes
warning	60–120 minutes
info	240+ minutes

Auto-Ticket Policy

Enable auto-ticket conservatively:

- **Enable for:** critical alerts (CPU >90%, disk >90%)
- **Disable for:** warning and info alerts — review manually, create tickets as needed

---

Chapter 30: Automation Profile Design

Design Principles

A well-designed automation profile answers:

1. What tasks need to run?
2. How often?
3. At what time to minimize disruption?
4. On which devices?
5. What should happen if something fails?

Profile Templates

Template 1: Nightly Security Maintenance (Standard)

```
Schedule: daily, 02:00
OS Patches: critical + security only
Disk: checkdisk = true, defrag = false
Maintenance: restore_point = true, delete_temp = true
Reboot: false
```

Template 2: Weekend Full Maintenance

```
Schedule: weekly, Sunday, 23:00
OS Patches: all categories including rollups
Software Patches: update_all = true
Disk: defrag = true (HDDs only), checkdisk = true
Maintenance: restore_point = true, delete_temp = true
Reboot: true
```

Template 3: Monthly Patch Tuesday

```
Schedule: monthly, second Tuesday, 22:00
OS Patches: critical + security + definitions
Disk: checkdisk = true
Maintenance: restore_point = true
Reboot: true
```

Common Mistakes to Avoid

1. **Do not schedule reboots on always-in-use devices.** Always confirm with clients first.
2. **Test on one device before fleet-wide rollout.**
3. **Exclude known-problematic packages** in the Software Patch exclusions list.
4. **Stagger schedules** across clients to avoid simultaneous API/database load.
5. **Review run history** after each profile run for failed tasks.

Chapter 31: User Roles and Permissions Matrix

Full Permissions Matrix

Feature / Action	Viewer	Technician	Admin	Superadmin
Dashboard				
View Dashboard Overview	Yes	Yes	Yes	Yes
Tickets				
View tickets	Yes	Yes	Yes	Yes
Create tickets	No	Yes	Yes	Yes
Update ticket status	No	Yes	Yes	Yes
Add comments (public)	No	Yes	Yes	Yes
Add comments (internal)	No	Yes	Yes	Yes
Delete tickets	No	No	Yes	Yes
Customers				
View customers	Yes	Yes	Yes	Yes
Create customers	No	Yes	Yes	Yes
Edit customers	No	Yes	Yes	Yes
Delete customers	No	No	Yes	Yes
Devices				
View device list	Yes	Yes	Yes	Yes
View device metrics	Yes	Yes	Yes	Yes
Alerts				
View alerts	Yes	Yes	Yes	Yes
Acknowledge alerts	No	Yes	Yes	Yes
Resolve alerts	No	Yes	Yes	Yes
Create/manage alert rules	No	Yes	Yes	Yes
App Center				
View software inventory	Yes	Yes	Yes	Yes
Network Discovery				

Feature / Action	Viewer	Technician	Admin	Superadmin
Run network scan	No	Yes	Yes	Yes
View scan results	Yes	Yes	Yes	Yes
Reports				
Generate reports	No	Yes	Yes	Yes
View/download report history	Yes	Yes	Yes	Yes
Billing				
View invoices	Yes	Yes	Yes	Yes
Create invoices	No	No	Yes	Yes
Update invoice status	No	No	Yes	Yes
Administration				
Access Admin page	No	No	Yes	Yes
View Audit Log	No	No	Yes	Yes
Manage users	No	No	Yes	Yes
Modify/delete superadmin account	No	No	No	CLI only
Automation				
View profiles	Yes	Yes	Yes	Yes
Create/edit profiles	No	Yes	Yes	Yes
Run profile now	No	Yes	Yes	Yes
Delete profiles	No	No	Yes	Yes
OS Patches				
View pending patches	Yes	Yes	Yes	Yes
Approve patches	No	Yes	Yes	Yes
Software Patches				
View software list	Yes	Yes	Yes	Yes
Check for updates	No	Yes	Yes	Yes
Disk Management				

Feature / Action	Viewer	Technician	Admin	Superadmin
View disk gauges	Yes	Yes	Yes	Yes
Run disk actions	No	Yes	Yes	Yes
Maintenance				
View maintenance page	Yes	Yes	Yes	Yes
Reboot/Shutdown devices	No	Yes	Yes	Yes
Run maintenance actions	No	Yes	Yes	Yes
Scripts				
View script library	Yes	Yes	Yes	Yes
Run scripts	No	Yes	Yes	Yes
Upload scripts	No	Yes	Yes	Yes
View run history	Yes	Yes	Yes	Yes

Chapter 32: Common Troubleshooting

Problem: Cannot log in — "Invalid credentials"

Cause: Wrong email, wrong password, or account is deactivated.

Steps:

1. Verify the email address — no typos, correct domain.
2. Check Caps Lock is off.
3. If recently changed password, try the new one.
4. If locked out, contact an administrator to check account status. If you are the only user and locked out:

```
UPDATE users SET is_active = true WHERE email = 'your@email.com';
```

Problem: Dashboard shows "API error: [message]"

Cause: Flask API at http://localhost:5000 is not responding.

Steps:

1. Check if API is running:

```
netstat -ano | findstr :5000
```

2. If nothing is on port 5000, start the API:

```
cd C:\RMM\RemoteManagementSystem\api
.\venv\Scripts\Activate.ps1
python app.py
```

3. If the API starts then immediately crashes, check the terminal output. Common causes:

• PostgreSQL not running (check port 5432 or Windows Services)
• Wrong `DATABASE_URL` in `.env`
• Missing Python dependencies (`pip install -r requirements.txt`)

Problem: Devices showing offline when they should be online

Cause: Agent stopped, device is off, or network issue.

Steps:

1. Check **Last Seen** timestamp. How long ago was it?
2. Under 5 minutes: likely a temporary network glitch — wait and recheck.
3. Over 10 minutes: physically check or remote into the device.
4. On the device, verify the agent is running:

```
Get-Process python -ErrorAction SilentlyContinue
```

5. If not running, restart:

```
cd C:\RMM\agent
.\venv\Scripts\Activate.ps1
python rmm_agent.py
```

6. Check `agent\rmm_agent.log` for error messages.

Problem: Agent won't register — "Registration failed"

Cause: Wrong API URL, wrong org_token, or API is not running.

Steps:

1. Open `agent\config.ini` on the managed device.
2. Verify `[api]` section:

```
[api]
url = http://YOUR_SERVER_IP:5000
org_token = YOUR_ORG_TOKEN
```

3. Test connectivity from the device:
   • Open a browser on that device and go to `http://YOUR_SERVER_IP:5000/api/health`
   • If you see a JSON response, the API is reachable

4. Verify `org_token` matches exactly — find it in the dashboard: **Admin** → **System Info** → **Agent Enrollment Token** → Reveal. Or check `ORG_REGISTRATION_TOKEN` in the API's `.env` file directly.

Problem: Scripts stuck in "queued" status

Cause: Agent offline, not polling, or Celery not running.

Steps:

1. Verify target device is online (green dot on Devices page).
2. Agent polls every 60 seconds — wait up to 2 minutes.
3. If still queued after 5 minutes, verify Celery worker is running.
4. Check Redis is running on port 6379:

```
Test-NetConnection -ComputerName localhost -Port 6379
```

Problem: Automation profile runs showing as "failed"

Cause: A task in the profile encountered an error.

Steps:

1. Go to **Maintenance** → Recent Maintenance Runs.
2. Find the failed run.
3. Note which profile and device.
4. Go to **Scripts** → Run History if the failure involves a script.
5. Read stderr for the error message.
6. Common causes:

• Device went offline during execution
• Agent lacked permissions for the task
• A specific patch installation failed

Problem: Cannot access Admin page — "Admin access required"

This is intentional. Admin page is restricted to admin role only.

To grant admin access (if you have database access):

```
UPDATE users SET role = 'admin' WHERE email = 'your@email.com';
```

Problem: Dashboard blank or session error after navigating

Cause: JWT token expired.

Steps:

1. Go to http://localhost:8501.

2. If you see the login screen, log in again.

Starting All Services — Quick Reference

```
# Verify PostgreSQL is running
Get-Service | Where-Object {$_.DisplayName -like "*postgresql*"}

# Verify Redis/Memurai is running
Test-NetConnection -ComputerName localhost -Port 6379

# Terminal 1 — Flask API
Set-Location C:\RMM\RemoteManagementSystem\api
.\venv\Scripts\Activate.ps1
python app.py

# Terminal 2 — Celery Worker
Set-Location C:\RMM\RemoteManagementSystem\api
.\venv\Scripts\Activate.ps1
celery -A tasks.celery_app worker --pool=solo -l info

# Terminal 3 — Celery Beat
Set-Location C:\RMM\RemoteManagementSystem\api
.\venv\Scripts\Activate.ps1
celery -A tasks.celery_app beat -l info

# Terminal 4 — Streamlit Dashboard
Set-Location C:\RMM\RemoteManagementSystem\dashboard
.\venv\Scripts\Activate.ps1
streamlit run app.py

# On each managed machine — Agent (as Administrator)
Set-Location C:\RMM\agent
.\venv\Scripts\Activate.ps1
python rmm_agent.py
```

APPENDIX A: GLOSSARY

Term	Definition
Agent	The Python program (`rmm_agent.py`) installed on managed Windows machines. Sends heartbeats every 60 seconds and executes remote commands.
Alert	An automatic notification generated when a device metric crosses a configured threshold.
Alert Rule	A configuration defining when alerts trigger: which metric, which threshold, with what severity.
API	Application Programming Interface. The Flask server at port 5000 that handles all data operations.
Automation Profile	A bundle of maintenance tasks (patching, disk, cleanup) scheduled to run automatically.
BAT	Windows Batch script file format.
Celery	A distributed task queue. Runs background tasks: alert evaluation, patch deployment, script dispatch.
Celery Beat	The Celery scheduler component. Triggers tasks on a schedule (every 60 seconds, daily profiles, etc.).
Compliance %	Percentage of managed devices that are fully patched and up to date.
Cooldown	Minimum time before an alert rule can fire again for the same device. Prevents alert flooding.
Critical	Highest alert severity. Immediate action required.
Dashboard	The Streamlit web interface at port 8501. Also specifically refers to the Overview page.
Defragmentation	Disk maintenance for HDDs that reorganizes fragmented files. Never run on SSDs.
Device	A managed machine with the RMM agent installed.
Exit Code	Number returned by a script when it finishes. 0 = success; non-zero = error.
Flask	Python web framework used to build the RMM API server.
Heartbeat	Regular check-in signal from the agent to the API every 60 seconds, reporting current metrics.
HDD	Hard Disk Drive — traditional spinning magnetic disk. Can benefit from defragmentation.

Term	Definition
Info	Lowest alert severity. Informational only; no immediate action needed.
Invoice	A billing document for a customer showing managed devices and the amount owed.
JWT	JSON Web Token. The authentication token used by this system.
KB	Knowledge Base number. Unique identifier for Windows patches (e.g., KB5034441).
Last Seen	Timestamp of the most recent heartbeat from a device's agent.
Memurai	Windows-native Redis-compatible server. Used as the Redis implementation on Windows.
Offline	A device whose agent has not sent a heartbeat recently.
Online	A device whose agent is actively sending heartbeats.
Org Token	Organization-level authentication token in `config.ini`, used during device registration. Must match the API's `ORG_REGISTRATION_TOKEN`.
OS	Operating System.
Patch	A software update addressing a security vulnerability, bug, or performance issue.
PostgreSQL	The relational database system storing all RMM data. Port 5432.
Priority	Ticket urgency level: low, medium, high, or critical.
PS1	PowerShell script file extension.
RBAC	Role-Based Access Control — permissions determined by user role.
Redis	In-memory data store used as the Celery message broker. Port 6379.
RMM	Remote Monitoring and Management.
Script	Code (PS1, BAT, PY, or SH) that can be executed remotely on a managed device.
Session	An active user login. Represented by a JWT token stored in browser memory.
Severity	Alert importance level: info, warning, or critical.

Term	Definition
SSD	Solid State Drive. Fast storage, no moving parts. Does not benefit from defragmentation.
Streamlit	Python web app framework used to build the RMM dashboard.
Technician	User role with full operational permissions but no user management or admin access.
Tier	Customer support level: standard, premium, or enterprise.
Timeout	Maximum time for a script to run before forced termination.
Token	See JWT. A cryptographic string proving a user is authenticated.
Viewer	User role with read-only access. Cannot make changes.
Warning	Medium severity alert. Indicates degraded performance needing attention soon.
Winget	Windows Package Manager. Used to install and update software on Windows.

APPENDIX B: QUICK REFERENCE CARDS

Quick Reference Card — New Employee (First Week)

Your daily starting routine:

1. Open http://localhost:8501 → log in
2. Check Dashboard stat cards — note any red numbers
3. Click **Overview** — scan the Device Health Map
4. Go to **Alerts** — acknowledge any new critical alerts
5. Go to **Tickets** — check for open or in-progress tickets
6. Respond to client calls — create tickets, investigate devices
7. Update ticket statuses throughout the day
8. Sign out when done (sidebar → Sign Out)

Creating a ticket: Tickets → + New Ticket → Title, Priority, Customer → Create Ticket

Finding a device: Devices → search for hostname → click to expand → read CPU/RAM/disk

Acknowledging an alert: Alerts → Active Alerts tab → expand alert → click Acknowledge

Updating ticket status: Tickets → expand ticket → Status dropdown → new status → Update Status

---

Quick Reference Card — IT Support Staff

Priority escalation guide:

- Client completely down / data at risk → **critical** ticket
- Significant business impact, service degraded → **high** ticket
- Non-urgent issue, one user affected → **medium** ticket
- Request, question, minor task → **low** ticket

Ticket lifecycle: `open` → `in_progress` → `resolved` (client confirms) → `closed`

When a client calls with a problem:

1. Create ticket first (documents the call)
2. Note everything they tell you in the description
3. Check Devices for their device health
4. Check Alerts for any related alerts
5. Add comments as you investigate
6. Update status at each stage

Checking software on a device: App Center → select device → search for application name

Checking if a device is online: Devices → find hostname → look at Status column (green dot = online)

Quick Reference Card — Technicians

Run a script on multiple devices: Scripts → Library → find script → expand → select devices → set timeout → Run

Approve OS patches: OS Patches → Pending Patches → check boxes → Approve Selected

Reboot a device: Maintenance → select device → check "I confirm..." → click Reboot

Create an alert rule: Alerts → Alert Rules → Create Alert Rule → fill form → Create Rule

Create an automation profile: Automation → Create / Edit Profile → fill fields → configure task columns → Save Profile

View disk health: Disk Management → select device → view gauges → run cleanup if needed

Check script output: Scripts → Run History → expand run entry → read stdout/stderr

Alert severity colors:

- Red = critical (immediate action)
- Amber = warning (attention needed)
- Blue = info (no action needed)

Ticket status colors:

- Red = open | Amber = in_progress | Green = resolved | Grey = closed

Script exit codes:

- 0 = SUCCESS | Non-zero = FAILED

Quick Reference Card — Managers

Generating a monthly client report: Reports → Generate → select template → select customer → set date range → Generate → Download

Report templates and what they cover:

- `device_summary` — device health and status overview
- `alert_summary` — all alerts by severity and device for the period
- `patch_summary` — patch compliance and deployment history
- `billing_summary` — invoices and billing totals

Checking outstanding invoices: Billing → view Outstanding metric at top → find unpaid invoices in the list

Creating an invoice: Billing → create invoice form → select customer → set period dates → enter device count and rate → Generate Invoice

Understanding the Dashboard for management:

- Stat cards: Total Devices, Online, Warning, Critical, Open Tickets

- Green numbers = healthy | Amber = attention needed | Red = action required

---

Quick Reference Card — Administrators

Access user management: Admin → Users tab

View audit log: Admin → Audit Log tab → filter by action type and date range

Check service health: Admin → System Info tab → Services card

Deactivate a departed employee: Admin → Users tab → find user → Edit → Deactivate → Save

Grant admin role (via database):

```
UPDATE users SET role = 'admin' WHERE email = 'user@company.com';
```

Service ports:

- Dashboard: http://localhost:8501
- Flask API: http://localhost:5000
- PostgreSQL: localhost:5432
- Redis: localhost:6379

Emergency service restart:

```
# Kill API: netstat -ano | findstr :5000 → taskkill /F /PID <PID>
# Kill dashboard: netstat -ano | findstr :8501 → taskkill /F /PID <PID>
```

Monthly security checklist:

- [ ] Review Audit Log for unexpected DELETE events
- [ ] Review Audit Log for unusual LOGIN IP addresses
- [ ] Verify all Users are current employees
- [ ] Check System Info → Services card for health
- [ ] Review Outstanding invoices in Billing
- [ ] Check Compliance % in OS Patches

Quick Reference Card — Developers

Dashboard entry point: `dashboard/app.py` **All pages:** `dashboard/pages/01_Dashboard.py` through `16_Scripts.py` **API client:** `dashboard/utils/api_client.py` **Auth utilities:** `dashboard/utils/auth.py` **CSS/UI helpers:** `dashboard/utils/styles.py` **Flask routes:** `api/routes/` **Models:** `api/models/` **Celery tasks:** `api/tasks/` **Agent main loop:** `agent/rmm_agent.py`

Brand colors:

- Primary: `#407E3C`
- Accent: `#5a9e56`

- White: `#FFFFFF`
- Danger: `#EF4444`
- Warning: `#F59E0B`
- Success: `#22C55E`

Auth pattern in dashboard pages:

```
from utils.auth import require_auth
client = require_auth() # Returns APIClient or redirects to login
```

API call pattern:

```
data, err = client.list_devices(per_page=100)
if err:
st.error(f"API error: {err}")
else:
devices = data.get("items", [])
```

Agent heartbeat interval: 60s (`config.ini → [agent] → heartbeat_interval`) **Software scan interval:** 21600s / 6h (`config.ini → [agent] → software_interval`) **Exit code convention:** 0 = SUCCESS, non-zero = FAILED

Run tests:

```
cd C:\RMM\RemoteManagementSystem\api
.\venv\Scripts\Activate.ps1
pytest tests/ -v
```

Quick Reference Card — WiFi Device Deployment

Deploy agent on a WiFi/LAN machine (Windows/Linux/macOS):

1. Get server LAN IP: Admin → System Info → Server IP Addresses
2. Copy `agent\` folder to target machine (USB, network share, or git clone)
3. On target machine:

```
cd C:\RMM\agent
python -m venv venv
.\venv\Scripts\Activate.ps1
pip install -r requirements.txt
python setup_agent.py 192.168.x.x <org_token>
```

Org token: Admin → System Info → Agent Enrollment Token → Reveal

4. Start the agent: `python rmm_agent.py`
5. Device appears in Devices tab within 60 seconds

Discover phones/IoT devices (no agent possible):

1. Network Discovery → enter subnet (e.g. `192.168.1.0/24`) → Scan Network

2. Review results — detected via OUI vendor, port probe, then hostname keywords (Samsung model names, brand names)
3. Click **Save All to Devices**
4. Devices appear in Devices → Android / iOS / Agentless tabs
5. System pings them every 5 minutes automatically

Quick Reference Card — Superadmin Emergency Recovery

Use this when: All regular admin accounts are locked out, forgotten, or deactivated and you cannot log in to the web interface.

Step 1 — Access the server machine directly (physical or remote desktop to the RMM server).

Step 2 — Open a terminal and reset the password:

```
Set-Location C:\RMM\RemoteManagementSystem\api
.\venv\Scripts\Activate.ps1
python reset_superadmin.py YourNewPassword123
```

Password must be at least 10 characters. No API restart needed.

Step 3 — Log in to the dashboard:

- URL: http://localhost:8501
- Email: `superadmin@rmm.local` (or whatever is set in `SUPERADMIN_EMAIL` in `.env`)
- Password: the one you just set

Step 4 — Recover access for regular admins:

- Go to Admin → Users tab
- Re-activate or reset passwords for admin accounts as needed

To permanently change superadmin credentials (recommended after install):

1. Edit `.env` — set `SUPERADMIN_EMAIL` and `SUPERADMIN_PASSWORD`
2. Restart the Flask API: `cd api ; python app.py`

End of RMM System Complete Handbook — Version 2.0

For support with this guide, contact your system administrator or development team.